ĐẠI HỌC QUỐC GIA TP.HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Hardware Implementation and Performance Evaluation of Viterbi Decoder for Digital Communication Systems

GVHD: TS. Phạm Quang Thái

SVTH: Lê Quý Dương 2113090

TP.Hồ Chí Minh, 2026

MỤC LỤC

LỜI MỞ ĐẦU 3
1 Đặt vấn đề 3
2 Mục tiêu của đề tài 3
LỜI CẢM ƠN 4

1 Cơ sở lý thuyết **5**
1.1 Hệ thống truyền thông số và mã hóa kênh (BPSK, AWGN, Eb/N0, BER) 5
1.1.1 Mô hình hệ thống 5
1.1.2 Điều chế BPSK 5
1.1.3 Kênh AWGN 5
1.1.4 Tỷ số Eb/N0 6
1.1.5 Tỷ lệ lỗi bit (BER) 6
1.2 Mã tích chập: tham số (n, k, K), đa thức sinh, bảng trạng thái, trellis 7
1.2.1 Khái niệm và các tham số 7
1.2.2 Bộ mã hóa $(2, 1, 3)$ với $G_1 = (1\ 1\ 1)$, $G_2 = (1\ 0\ 1)$ 8
1.2.3 Bảng trạng thái và sơ đồ trạng thái 9
1.2.4 Sơ đồ trellis 10
1.2.5 Nhận xét 10
1.3 Thuật toán Viterbi: branch metric, ACS, survivor path, traceback 10
1.3.1 Ý tưởng 10
1.3.2 Branch metric (BM) 10
1.3.3 Path metric và phép Add-Compare-Select (ACS) 11
1.3.4 Bộ nhớ survivor và truy hồi (traceback) 12
1.3.5 Ví dụ giải mã có một lỗi 12
1.3.6 Kiến trúc phần cứng tổng quát 12
1.4 Giải mã quyết định cứng và quyết định mềm 12
1.4.1 So sánh giải mã quyết định cứng và quyết định mềm 12
1.4.2 Ảnh hưởng đến kiến trúc phần cứng 13
1.5 Lượng tử hóa thông tin mềm và traceback deepth 15
1.5.1 Lượng tử hóa thông tin mềm 15
1.5.2 Traceback depth 15

2 Mô hình tham chiếu Matlab **16**
2.1 Thiết kế mô hình . . . 16
2.1.1 Sơ đồ khối . . . 16
2.1.2 Các tham số mô phỏng . . . 17
2.1.3 Phương pháp tính nhiễu . . . 18
2.2 Kết quả BER: Hệ thống không mã hóa vs Có mã hóa (Hard) . . . 18
2.2.1 Biểu đồ . . . 18
2.2.2 Nhận xét . . . 19
2.3 So sánh giải mã Hard-decision và Soft-decision . . . 20
2.4 Tổng hợp Trade-off . . . 20
2.5 Nhận xét . . . 21
2.6 Kết luận Chương 2 . . . 21

3 Thiết kế phần cứng RTL **22**
3.1 Kiến trúc tổng quát của bộ giải mã Viterbi . . . 22
3.2 Chi tiết thiết kế các khối chức năng . . . 23
3.2.1 Khối tính toán Metric nhánh (BMU - Branch Metric Unit) . . . 23
3.2.2 Khối cộng - so sánh - chọn lọc (ACSU - Add-Compare-Select Unit) . . . 25
3.2.3 Khối chuẩn hóa Path Metric (Normalization) . . . 26
3.2.4 Bộ nhớ lưu vết (Survivor Memory) . . . 27
3.2.5 Khối truy hồi (Traceback Unit) . . . 29
3.3 Tham số và giao diện RTL . . . 30
3.3.1 Bảng tham số . . . 30
3.3.2 Giao diện tín hiệu . . . 31
3.3.3 Giao thức (Protocol) và Định thời (Timing) . . . 31
3.3.4 Quy ước thiết kế . . . 32
3.4 Phiên bản Hard-decision . . . 32
3.4.1 Điểm khác biệt về BMU (Branch Metric Unit) . . . 32
3.4.2 Điểm khác biệt về độ rộng Metric (Metric Width) . . . 33
3.4.3 Điểm khác biệt về kỹ thuật chuẩn hóa (Normalization) . . . 33
3.4.4 Các tham số dùng chung (Shared Parameters) . . . 33

4 Kiểm chứng RTL với MATLAB **35**
4.1 Môi trường kiểm chứng . . . 35
4.2 Kế hoạch kiểm thử (Test Plan) . . . 37
4.3 Kết quả kiểm chứng . . . 38
4.3.1 Kết quả Unit Test (42 bits) . . . 38
4.3.2 Kết quả so sánh RTL vs MATLAB (File-based Cosim) . . . 39

LỜI MỞ ĐẦU

1 Đặt vấn đề

Trong các hệ thống truyền thông số, tín hiệu khi truyền qua kênh luôn chịu ảnh hưởng của nhiễu, làm cho một số bit ở phía thu bị nhận sai. Để giảm tỷ lệ lỗi bit (BER) mà không phải tăng công suất phát, các hệ thống sử dụng kỹ thuật mã hóa sửa lỗi kênh truyền (FEC), tức là thêm vào dữ liệu những bit dư thừa có quy luật trước khi truyền đi. Mã tích chập (convolutional code) là một trong những phương pháp như vậy, được dùng rộng rãi trong truyền thông không dây và vệ tinh nhờ cấu trúc đơn giản và khả năng sửa lỗi tốt.

Phía thu, thuật toán Viterbi là phương pháp giải mã mã tích chập theo tiêu chí hợp lý cực đại (Maximum Likelihood): thuật toán tìm trong sơ đồ trellis đường đi có tổng độ sai khác nhỏ nhất so với chuỗi nhận được, từ đó khôi phục chuỗi bit gốc. Tuy nhiên, bộ giải mã Viterbi đòi hỏi tính toán nhiều (các phép cộng, so sánh, chọn và lưu vết đường đi), nên khi cài đặt trên phần cứng phải cân nhắc giữa độ chính xác giải mã và chi phí phần cứng như tài nguyên, độ trễ, tốc độ xử lý.

2 Mục tiêu của đề tài

Đề tài Hardware Implementation and Performance Evaluation of Viterbi Decoder for Digital Communication Systems hướng đến phát triển bộ giải mã Viterbi bằng ngôn ngữ mô tả phần cứng ở mức RTL, đồng thời đánh giá sự đánh đổi giữa hiệu năng BER và độ phức tạp phần cứng. Cụ thể:

Xây dựng mô hình tham chiếu (MATLAB hoặc Python) cho hệ thống gồm mã hóa tích chập, điều chế BPSK, kênh AWGN và giải mã Viterbi.

Thiết kế và kiểm chứng RTL của bộ giải mã Viterbi so với mô hình tham chiếu.

So sánh giải mã quyết định cứng (hard-decision) và quyết định mềm (soft-decision).

Khảo sát ảnh hưởng của việc lượng tử hóa thông tin mềm và của độ sâu truy hồi (traceback depth).

Đánh giá độ trễ, thông lượng, tần số hoạt động tối đa (Fmax) và tài nguyên phần cứng của thiết kế.

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến TS.Phạm Quang Thái đã tận tình hướng dẫn, định hướng và góp ý cho em trong suốt quá trình thực hiện đề tài.

Em xin cảm ơn quý thầy cô Khoa Điện-Điện Tử Trường Đại học Bách Khoa - DHQG-TPHCM đã truyền đạt cho em những kiến thức nền tảng cần thiết. Em cũng xin cảm ơn gia đình và bạn bè đã động viên, hỗ trợ em trong thời gian qua.

Do thời gian và kiến thức còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Em rất mong nhận được ý kiến đóng góp của quý thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 10 năm 2026

Sinh viên thực hiện
Lê Quý Dương

Chương 1

Cơ sở lý thuyết

1.1 Hệ thống truyền thông số và mã hóa kênh (BPSK, AWGN, Eb/N0, BER)

1.1.1 Mô hình hệ thống

Hệ thống truyền thông số trong đề tài gồm các khối nguồn bit, mã hóa tích chập, điều chế BPSK, kênh AWGN, giải điều chế và giải mã Viterbi như minh họa trong Hình 1. Sau khi truyền qua kênh nhiễu, bộ giải mã Viterbi khai thác cấu trúc của chuỗi mã để khôi phục dữ liệu ban đầu và là đối tượng thiết kế phần cứng chính của đề tài.



Hình 1: *Sơ đồ khối hệ thống truyền thông số*

1.1.2 Điều chế BPSK

Trong đề tài, điều chế BPSK sử dụng quy ước ánh xạ bit 0 thành ký hiệu -1 và bit 1 thành ký hiệu $+1$, được biểu diễn bởi công thức:

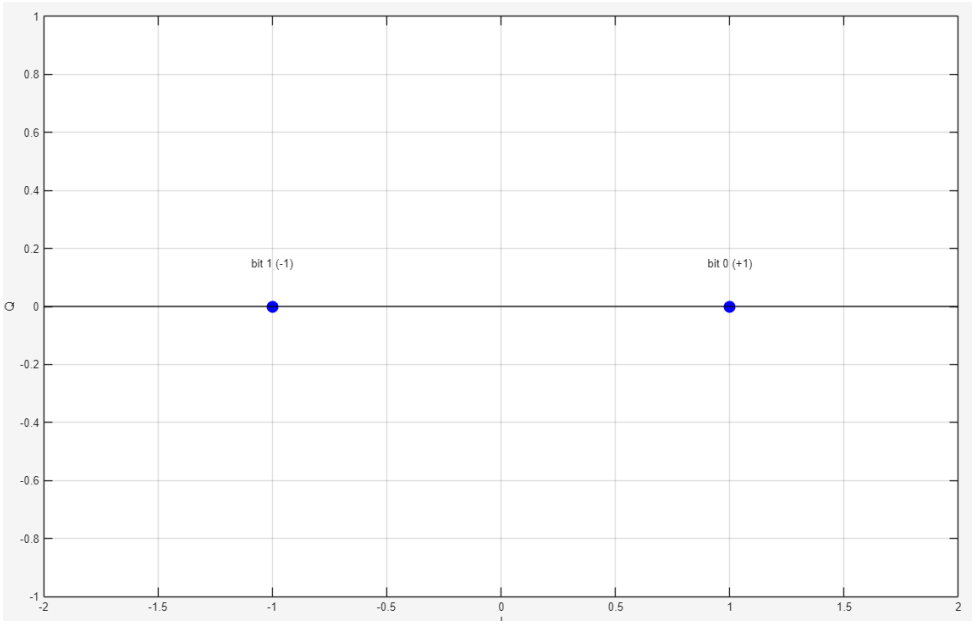
$$x = 2v - 1, \quad v \in \{0, 1\}.$$

Quy ước này được sử dụng thống nhất trong mô hình MATLAB và quá trình mô phỏng hệ thống.

1.1.3 Kênh AWGN

Kênh truyền được mô hình hóa bằng kênh nhiễu Gauss trắng cộng tính (AWGN), với tín hiệu thu được biểu diễn như sau:

$$y = x + n,$$



Hình 2: *Chòm sao BPSK*

trong đó x là tín hiệu phát, y là tín hiệu thu và n là nhiễu Gauss có trung bình bằng 0, phương sai:

$$\sigma^2 = \frac{N_0}{2},$$

với N_0 là mật độ phổ công suất nhiễu.

1.1.4 Tỷ số Eb/N0

Để đánh giá hiệu năng hệ thống, tỷ số E_b/N_0 được sử dụng làm đại lượng biểu diễn mức năng lượng trên mỗi bit thông tin so với mật độ phổ công suất nhiễu. Với mã tốc độ $R = k/n$, mối quan hệ giữa năng lượng bit mã và năng lượng bit thông tin được xác định như sau:

$$E_s = R \cdot E_b \implies \frac{E_s}{N_0} = R \cdot \frac{E_b}{N_0} \tag{1.1}$$

Khi chuẩn hóa năng lượng ký hiệu ($E_s = 1$), phương sai nhiễu AWGN được xác định theo:

$$\sigma^2 = \frac{N_0}{2} = \frac{1}{2 \cdot R \cdot \frac{E_b}{N_0}} \tag{1.2}$$

Lưu ý: Cần sử dụng đúng hệ số tốc độ mã (R) khi tính phương sai nhiễu để bảo đảm tính chính xác của kết quả mô phỏng BER.

1.1.5 Tỷ lệ lỗi bit (BER)

Tỷ lệ lỗi bit (BER) được sử dụng để đánh giá hiệu năng của hệ thống truyền thông. Đối với BPSK không mã hóa với quyết định cứng trên kênh AWGN, BER lý thuyết được

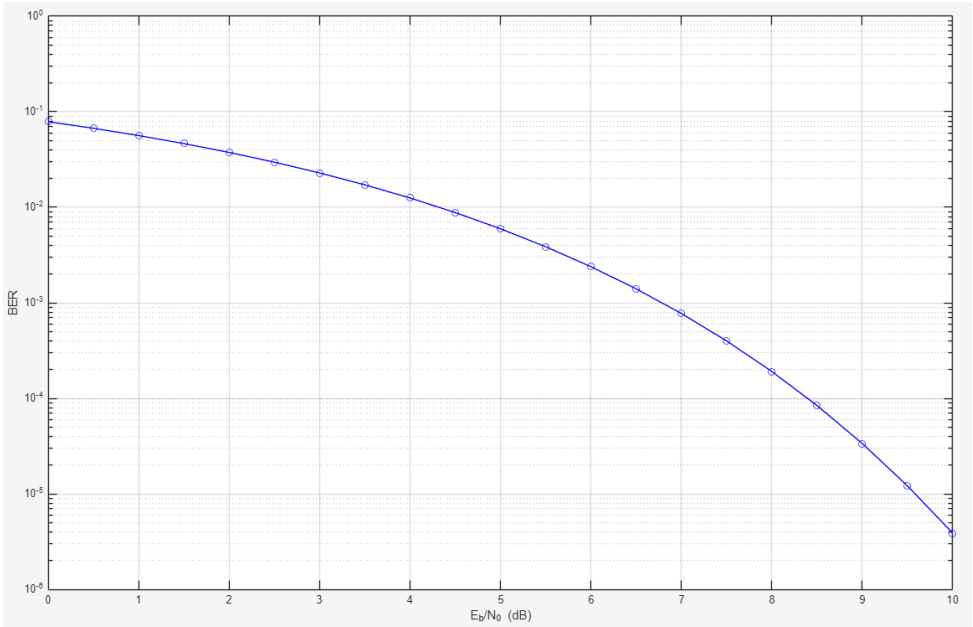
xác định theo công thức:

$$\text{BER} = Q\left(\sqrt{\frac{2E_b}{N_0}}\right) \tag{1.3}$$

với hàm Q được định nghĩa là:

$$Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^\infty e^{-\frac{t^2}{2}} dt$$

Công thức này được sử dụng làm đường tham chiếu (baseline) để so sánh với kết quả BER thu được từ mô phỏng MATLAB ở Chương 2.



Hình 3: *BER lý thuyết*

Đường cong này được dùng làm chuẩn tham chiếu (*uncoded*). Hiệu quả của mã hóa kênh được thể hiện bằng độ lệch giữa đường BER của hệ thống có mã và đường tham chiếu tại cùng E_b/N_0 (gọi là độ lợi mã hóa, tính bằng dB). Để ước lượng BER tin cậy bằng mô phỏng, cần truyền đủ nhiều bit sao cho số bit sai đếm được đủ lớn (thông thường vài chục đến hàng trăm lỗi).

1.2 Mã tích chập: tham số (n, k, K), đa thức sinh, bảng trạng thái, trellis

1.2.1 Khái niệm và các tham số

Mã tích chập là một dạng mã tuyến tính, trong đó chuỗi bit mã tại mỗi thời điểm không chỉ phụ thuộc vào bit vào hiện tại mà còn phụ thuộc vào một số bit vào trước đó. Bộ mã hóa có thể xem như một bộ lọc số tuyến tính, bất biến theo thời gian trên trường

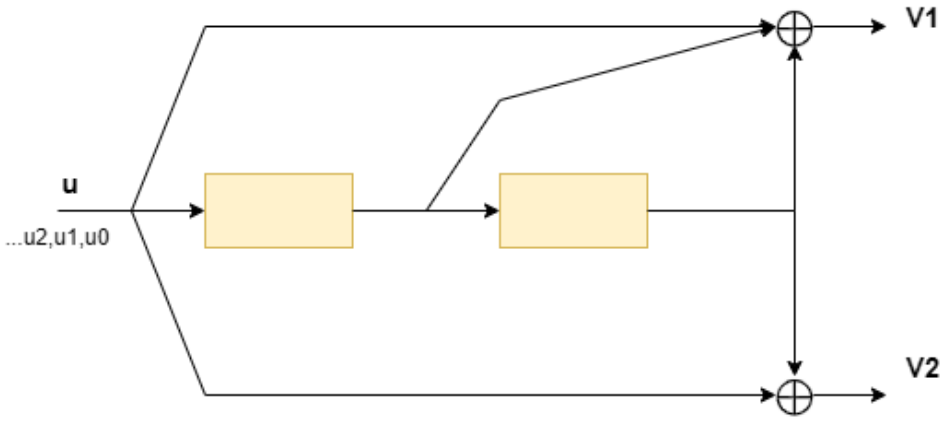
nhị phân: mỗi ngõ ra là phép tích chập (modulo 2) của chuỗi vào với một đa thức sinh. Từ đó có tên "mã tích chập". Bộ mã hóa được đặc trưng bởi các tham số:

Bảng 1: Các tham số cơ bản của mã tích chập

Ký hiệu	**Ý nghĩa**	**Giá trị trong đề tài**
k	Số bit vào mỗi lần	1
n	Số bit ra mỗi lần	2
$R = k/n$	Tốc độ mã	1/2
$m = K - 1$	Số ô nhớ (thanh ghi dịch)	2
K	Độ dài ràng buộc (số bit ảnh hưởng đến một cặp ngõ ra, kể cả bit hiện tại)	3
Số trạng thái	$2^{K-1} = 2^m$	4

Với $K = 3$, mỗi cặp ngõ ra phụ thuộc vào bit hiện tại u_0 và hai bit trước đó u_1, u_2. K càng lớn thì khả năng sửa lỗi càng tốt nhưng số trạng thái, và do đó độ phức tạp của bộ giải mã Viterbi, tăng theo hàm mũ.

1.2.2 Bộ mã hóa $(2, 1, 3)$ với $G_1 = (1\ 1\ 1)$, $G_2 = (1\ 0\ 1)$



Hình 4: *Sơ đồ bộ mã hóa*

Đề tài sử dụng hai đa thức sinh ở dạng nhị phân (hệ số của u_0, u_1, u_2):

$$G_1 = (1\ 1\ 1), \quad \text{tương ứng } 1 + x + x^2 \tag{1.4}$$

$$G_2 = (1\ 0\ 1), \quad \text{tương ứng } 1 + x^2 \tag{1.5}$$

Hai ngõ ra được tính theo công thức:

$$V^1 = u_0 \oplus u_1 \oplus u_2 \tag{1.6}$$

$$V^2 = u_0 \oplus u_2 \tag{1.7}$$

hay tổng quát $V^j = \sum u_i \cdot G_i^j \pmod 2$, tức $V^j = U * G_j$. Chuỗi ngõ ra được ghép xen kẽ:

$V = (V_0^1, V_0^2, V_1^1, V_1^2, \dots)$. Quy ước của đề tài: V^1 xuất trước V^2, và thanh ghi ban đầu ở trạng thái 00.

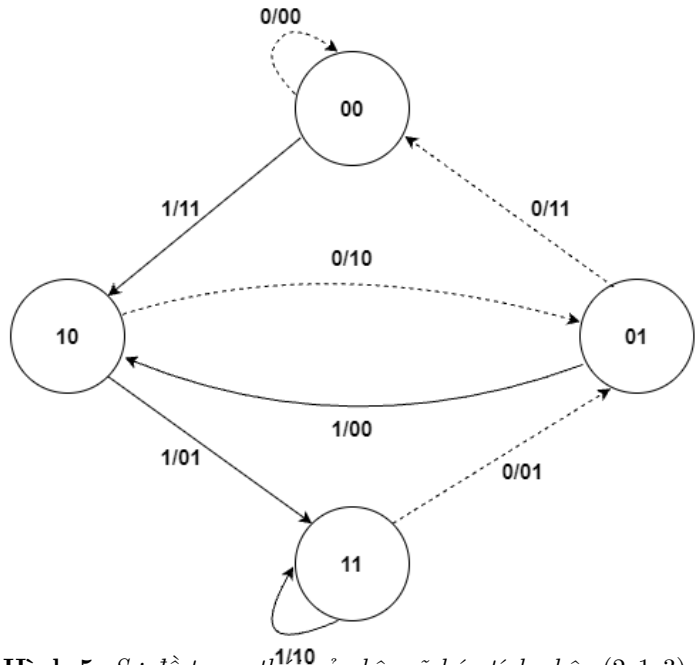
1.2.3 Bảng trạng thái và sơ đồ trạng thái

Bộ mã hóa là một máy trạng thái hữu hạn dạng Mealy. Trạng thái được định nghĩa là nội dung hai ô nhớ, ký hiệu $S = (u_1\ u_2)$. Khi có bit vào u, ngõ ra là (V^1, V^2) và trạng thái kế tiếp là $S' = (u, u_1)$, tức bit vào mới được dịch vào ô đầu. Bảng 2 liệt kê toàn bộ 8 chuyển trạng thái.

Ví dụ ở dòng bit vào 1, trạng thái 10: $V^1 = 1 \oplus 1 \oplus 0 = 0$ và $V^2 = 1 \oplus 0 = 1$, ngõ ra là 01, trạng thái kế là 11.

Bảng 2: Bảng trạng thái của bộ mã hóa $(2, 1, 3)$

Bit vào u	**Trạng thái hiện tại** (u_1u_2)	**Ngõ ra** V^1V^2	**Trạng thái kế** (uu_1)
0	00	00	00
0	01	11	00
0	10	10	01
0	11	01	01
1	00	11	10
1	01	00	10
1	10	01	11
1	11	10	11



Hình 5: *Sơ đồ trạng thái của bộ mã hóa tích chập* $(2, 1, 3)$

1.2.4 Sơ đồ trellis

Sơ đồ trellis biểu diễn các trạng thái và chuyển trạng thái của bộ mã hóa theo thời gian. Cấu trúc này là cơ sở để thuật toán Viterbi đánh giá các đường đi và lựa chọn đường có metric tốt nhất. Sơ đồ trellis và vai trò của nó trong kiến trúc giải mã sẽ được trình bày ở Chương 3.

1.2.5 Nhận xét

Phần dư thừa trong chuỗi mã (cứ 1 bit vào sinh ra 2 bit ra) chính là thứ giúp bộ giải mã sửa lỗi. Do tính chất tuyến tính và ràng buộc $K = 3$, chỉ một số chuỗi mã nhất định là hợp lệ, và nhiễu thường đẩy chuỗi nhận được ra khỏi tập chuỗi hợp lệ. Bộ giải mã tìm chuỗi hợp lệ gần nhất.

Bộ mã hóa rất đơn giản về phần cứng (2 thanh ghi và 3 cổng XOR), trong khi độ phức tạp tập trung ở phía giải mã. Vì vậy phần thiết kế RTL của đề tài tập trung vào bộ giải mã Viterbi, còn bộ mã hóa chủ yếu dùng để tạo dữ liệu kiểm thử.

1.3 Thuật toán Viterbi: branch metric, ACS, survivor path, traceback

1.3.1 Ý tưởng

Mỗi chuỗi bit thông tin tương ứng với đúng một đường đi trong trellis. Nhiễu làm chuỗi nhận được không còn nằm trên đường nào hợp lệ, nên bộ giải mã phải chọn đường đi "gần" chuỗi nhận được nhất. Đó là giải mã hợp lý cực đại (*Maximum Likelihood, ML*). Với kênh AWGN, "gần nhất" nghĩa là sai khác nhỏ nhất: khoảng cách Hamming nếu dùng quyết định cứng, khoảng cách Euclid nếu dùng quyết định mềm.

Duyệt vét cạn sẽ phải so sánh 2^N đường với chuỗi dài N bit. Thuật toán Viterbi tránh việc này nhờ nhận xét: tại mỗi nút, chỉ cần giữ lại một đường đến nút đó có tổng sai khác nhỏ nhất (gọi là đường sống sót, *survivor*), vì các đường bị loại không thể trở thành đường tốt nhất về sau. Số phép tính chỉ còn tỷ lệ với $N \times 2^{K-1}$ thay vì tăng theo hàm mũ của N.

1.3.2 Branch metric (BM)

Branch metric (BM) biểu diễn độ sai khác giữa dữ liệu nhận được và cặp bit đầu ra tương ứng với một nhánh trellis. Với quyết định cứng, BM được tính bằng khoảng cách Hamming theo công thức:

$$BM = d_H(r, e) \tag{1.8}$$

Trong đó r là cặp bit nhận được và e là cặp bit đầu ra ứng với nhánh trellis. Trong phần cứng, phép tính này có thể được thực hiện bằng các phép XOR kết hợp với phép cộng số bit khác nhau. Đối với quyết định mềm, cách tính BM sẽ được trình bày ở phần so sánh hai phương pháp giải mã.

1.3.3 Path metric và phép Add-Compare-Select (ACS)

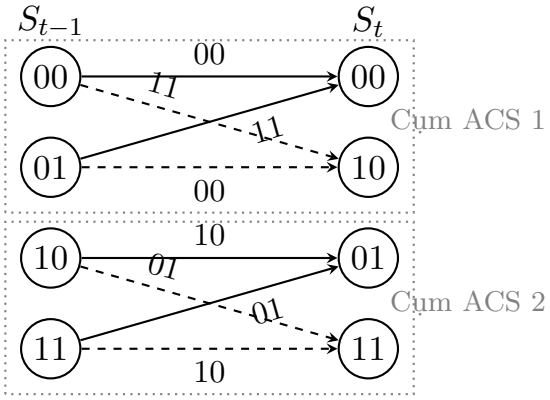
Path metric $\mathrm{PM}_t(s)$ là tổng branch metric lớn nhất của các đường kết thúc ở trạng thái s tại thời điểm t. Tại mỗi bước, mỗi trạng thái đích s' có đúng hai trạng thái đi tới (hai nhánh vào), và ACS gồm ba thao tác:

- **Add**: cộng BM của từng nhánh vào PM của trạng thái nguồn.
- **Compare**: so sánh hai tổng.
- **Select**: chọn tổng lớn hơn làm PM mới, đồng thời lưu lại nhánh được chọn (bit quyết định).

$$\mathrm{PM}_t(s') = \max\left\{\mathrm{PM}_{t-1}(s_a) + \mathrm{BM}(s_a \to s'), \mathrm{PM}_{t-1}(s_b) + \mathrm{BM}(s_b \to s')\right\} \tag{1.9}$$

Khi hai tổng bằng nhau (hòa), ta cần một quy ước cố định. Đề tài chọn nhánh từ trạng thái nguồn có giá trị nhỏ hơn. Khởi tạo: $\mathrm{PM}_0(00) = 0$ và PM_0 của các trạng thái còn lại bằng một số lớn (đại diện cho vô cùng), vì bộ mã hóa bắt đầu ở trạng thái 00.

Từ Bảng 2, bốn trạng thái đích chia thành hai nhóm **"cánh bướm"**: $\{00, 01\}$ cùng đi tới $\{00, 10\}$, và $\{10, 11\}$ cùng đi tới $\{01, 11\}$. Cấu trúc này giúp việc tổ chức phần cứng ACS đối xứng và dễ song song hóa.



Hình 6: *Cấu trúc cánh bướm trong phép tính ACS*

1.3.4 Bộ nhớ survivor và truy hồi (traceback)

Trong mỗi bước ACS, bộ giải mã lưu một bit quyết định cho mỗi trạng thái đích để ghi nhận trạng thái nguồn được chọn. Vì mỗi trạng thái đích có hai trạng thái nguồn khả dĩ, một bit quyết định đủ để xác định nguồn được chọn giữa hai khả năng (a, b). Quá trình traceback sử dụng các bit quyết định đã lưu để truy hồi ngược đường sống sót và khôi phục chuỗi bit thông tin.

1.3.5 Ví dụ giải mã có một lỗi

Ví dụ giải mã được sử dụng để minh họa quá trình khôi phục chuỗi bit thông tin bằng thuật toán Viterbi. Thay vì liệt kê toàn bộ path metric theo từng thời điểm, tính đúng đắn của thiết kế phần cứng sẽ được kiểm chứng thông qua waveform mô phỏng, bằng cách đối chiếu chuỗi bit đầu vào với chuỗi bit sau giải mã.

1.3.6 Kiến trúc phần cứng tổng quát

Từ thuật toán trên, bộ giải mã Viterbi gồm bốn khối chức năng:

- **BMU (Branch Metric Unit)**: tính các branch metric từ cặp bit (hoặc giá trị mềm) nhận được.
- **ACSU (Add-Compare-Select Unit)**: cập nhật path metric và sinh bit quyết định cho 4 trạng thái.
- **Bộ nhớ survivor**: lưu các bit quyết định theo thời gian.
- **Khối traceback**: truy hồi và xuất bit giải mã.

Kiến trúc này là cơ sở cho thiết kế RTL của bộ giải mã Viterbi; chức năng và cách hiện thực từng khối sẽ được trình bày chi tiết ở Chương 3.

1.4 Giải mã quyết định cứng và quyết định mềm

1.4.1 So sánh giải mã quyết định cứng và quyết định mềm

Trong giải mã Viterbi, quyết định cứng (Hard Decision) và quyết định mềm (Soft Decision) khác nhau ở thông tin đầu vào và cách tính branch metric (BM).

Quyết định cứng: Tín hiệu thu được so sánh với ngưỡng 0 để xác định bit 0 hoặc 1. Với ánh xạ BPSK đã chọn, quy tắc quyết định là:

$$\hat{v} = \begin{cases} 0, & y \geq 0 \\ 1, & y < 0 \end{cases} \tag{1.10}$$

Branch metric được tính bằng khoảng cách Hamming giữa cặp bit nhận được và cặp bit đầu ra của nhánh trellis. Phương pháp này có ưu điểm là phần cứng đơn giản, nhưng loại bỏ thông tin về độ tin cậy của tín hiệu.

Quyết định mềm: Thay vì chỉ sử dụng bit đã phân ngưỡng, bộ giải mã nhận giá trị tín hiệu sau lượng tử hóa r_i. Với kênh AWGN, tiêu chí hợp lý cực đại tương đương việc chọn đường đi có tổng bình phương khoảng cách Euclid nhỏ nhất. Tuy nhiên, để tránh phép bình phương trong phần cứng, branch metric mềm thường được tính dựa trên tích vô hướng (dot product) giữa tín hiệu thu và ký hiệu phát dự kiến:

$$BM_{soft} = (2e_1 - 1) \cdot r_0 + (2e_0 - 1) \cdot r_1 \tag{1.11}$$

Trong đó: e_1, e_0 là cặp bit ngõ ra dự kiến của nhánh, r_0, r_1 là giá trị mềm nhận được. Phương pháp này tận dụng thêm thông tin về độ tin cậy, nhờ đó cải thiện hiệu năng BER, nhưng yêu cầu độ rộng dữ liệu và khối tính toán lớn hơn.

Để thấy rõ sự khác biệt, xét ví dụ với lượng tử hóa $q = 3$ bit. Giả sử cặp ký hiệu thu được biểu diễn dưới dạng số có dấu $y = (+1; -3)$.

- Với quyết định cứng, bộ giải điều chế cho ra cặp bit 01.

- Với quyết định mềm, áp dụng công thức (1.11) ta có bảng so sánh.

Áp dụng công thức tính BM cứng và mềm, ta có bảng so sánh sau:

Bảng 3: So sánh branch metric cứng và mềm cho cặp thu $y = (+1; -3)$

Ngõ ra nhánh e	**BM cứng (Hamming với** 01**)**	**BM mềm theo** (15)**,** $z = (3; 7)$
00	1	$-1 - (-3) = 2$
01	0	$-1 + (-3) = -4$
10	2	$1 - (-3) = 4$
11	1	$1 + (-3) = -2$

Như minh họa trong Bảng 3, quyết định mềm có thể phân biệt mức độ tin cậy giữa các ký hiệu thu, trong khi quyết định cứng chỉ sử dụng giá trị bit sau phân ngưỡng. Vì vậy, hai phương pháp tạo ra sự đánh đổi giữa hiệu năng giải mã và chi phí phần cứng.

1.4.2 Ảnh hưởng đến kiến trúc phần cứng

Chuyển từ quyết định cứng sang quyết định mềm làm thay đổi khối BMU (tính toán metric nhánh) và độ rộng đường dữ liệu (datapath). Do giá trị Branch Metric trở nên lớn hơn và có dấu, độ rộng của Path Metric (thanh ghi) và mạch cộng trong khối ACSU cũng phải tăng lên tương ứng để tránh tràn số. Tuy nhiên, cấu trúc logic điều khiển của

ACSU, bộ nhớ Survivor (vẫn lưu 1 bit quyết định/trạng thái) và khối Traceback được giữ nguyên về mặt kiến trúc.

Bảng 4: So sánh ảnh hưởng kiến trúc phần cứng

Tiêu chí	**Hard Decision**	**Soft Decision (q bit)**	**Ảnh hưởng đến Fmax/Resources**
Ngõ vào decoder mỗi bước	2 bit	$2q$ bit	Soft Decision cần bus dữ liệu rộng hơn, có thể tăng tài nguyên kết nối và thanh ghi đầu vào.
Branch metric	Khoảng cách Hamming, từ 0 đến 2	Phụ thuộc giá trị tín hiệu sau lượng tử hóa	Soft Decision cần mạch tính toán BM phức tạp hơn, có thể tăng diện tích logic và độ trễ đường dữ liệu.
Độ rộng BM	2 bit	Phụ thuộc q và cách biểu diễn BM	BM rộng hơn có thể làm tăng số bit của bộ cộng và mạch xử lý metric.
Độ rộng path metric	Nhỏ	Thường lớn hơn	Path metric rộng hơn làm tăng tài nguyên của thanh ghi, bộ cộng và bộ so sánh; có thể ảnh hưởng F_{max}.
Survivor memory và traceback	1 bit quyết định/trạng thái/bước	Không đổi nếu vẫn lưu một bit quyết định cho mỗi trạng thái	Dung lượng survivor memory cơ bản không đổi khi số trạng thái và độ sâu truy hồi giữ nguyên.
Độ lợi mã hóa	Mức tham chiếu ($0\ dB$)	Lớn hơn khoảng $1.8\ dB$	Soft Decision cải thiện khả năng giải mã nhưng phải đánh đổi bằng tài nguyên phần cứng.

Lưu ý: Các nhận xét về F_{max} và tài nguyên là đánh giá định tính theo cấu trúc thiết kế; kết quả tổng hợp hoặc triển khai thực tế mới xác định được mức ảnh hưởng cụ thể.

1.5 Lượng tử hóa thông tin mềm và traceback deepth

1.5.1 Lượng tử hóa thông tin mềm

Tín hiệu thu từ kênh AWGN có giá trị liên tục, trong khi phần cứng số xử lý dữ liệu với độ rộng bit hữu hạn. Vì vậy, tín hiệu cần được lượng tử hóa trước khi đưa vào bộ giải mã Soft Decision. Số bit lượng tử hóa ảnh hưởng đến độ chính xác biểu diễn tín hiệu, hiệu năng BER và tài nguyên phần cứng. Trong cấu hình khảo sát của đề tài, tín hiệu tương tự sau khi qua kênh AWGN được lượng tử hóa thành 8 mức (3-bit). Mặc dù vậy, để thuận tiện cho việc kết nối và đồng bộ hóa phần cứng, đường bus dữ liệu đầu vào của khối RTL được thiết kế với độ rộng 8-bit (`SOFT_W` = 8)

1.5.2 Traceback depth

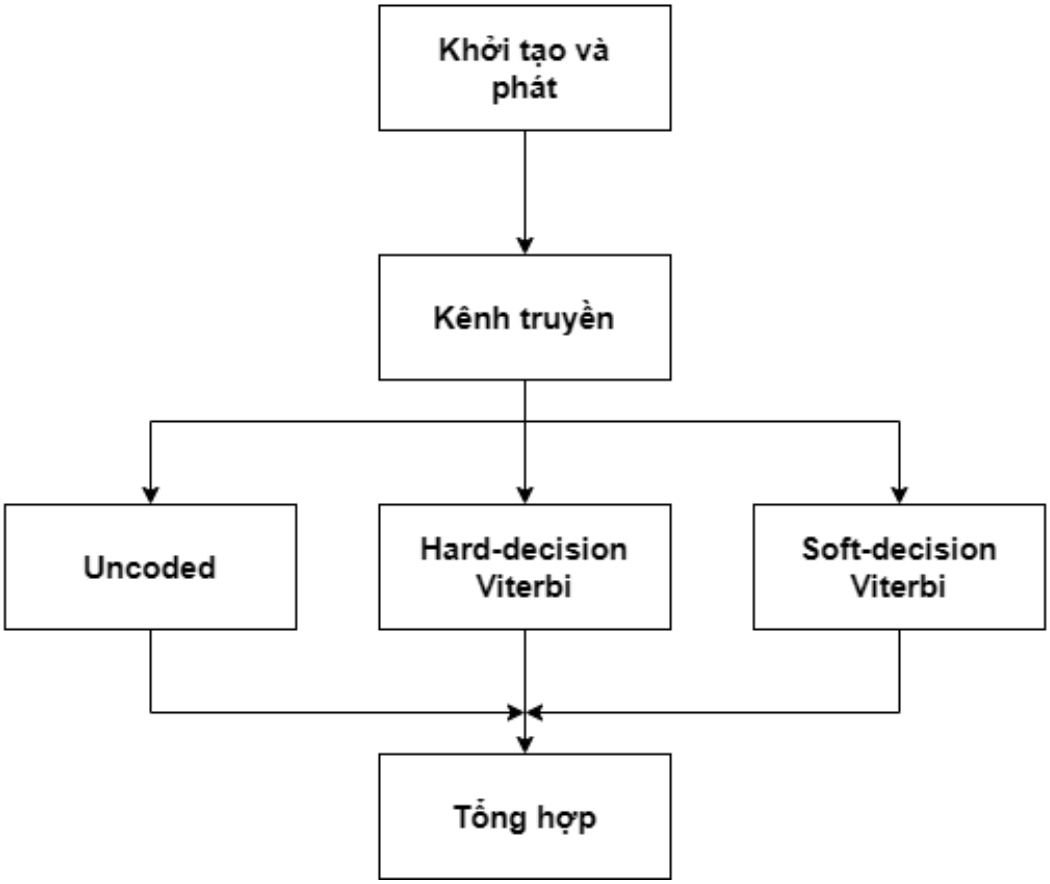
Traceback depth là số bước thời gian được sử dụng khi truy hồi đường sống sót để khôi phục chuỗi bit thông tin. Với mã tích chập có độ dài ràng buộc (K=3), đề tài chọn độ sâu truy hồi bằng (4K=12) symbol theo quy tắc thiết kế được sử dụng. Tăng traceback depth thường giúp giảm lỗi truy hồi do đường sống sót chưa hội tụ, nhưng đồng thời làm tăng dung lượng bộ nhớ survivor và độ trễ giải mã. Vì vậy, cần lựa chọn độ sâu phù hợp để cân bằng giữa hiệu năng BER, tài nguyên phần cứng và latency.

Chương 2

Mô hình tham chiếu Matlab

2.1 Thiết kế mô hình

2.1.1 Sơ đồ khối



Hình 7: *Sơ đồ khối mô hình mô phỏng hệ thống truyền thông sử dụng mã hóa tích chập.*

- **Khởi tạo và phát:** Tạo ngẫu nhiên 10^6 bit, thêm 2 bit kết thúc (*tail bits*) để đưa bộ mã hóa về trạng thái 00. Thực hiện mã hóa tích chập $(2, 1, 3)$ với đa thức sinh

[7 5] và điều chế BPSK ($0 \to +1, 1 \to -1$).

- **Kênh truyền:** Mô phỏng nhiễu Gauss trắng cộng tính (AWGN) bằng cách cộng thêm nhiễu ngẫu nhiên có phương sai $\sigma^2 = \frac{1}{2 \cdot R \cdot E_b/N_0}$.
- **Nhánh Uncoded:** Tín hiệu thu được đưa qua bộ cắt ngưỡng cứng (ngưỡng 0) để đưa ra quyết định bit, sau đó đếm số bit lỗi để làm mốc tham chiếu.
- **Nhánh Hard-decision Viterbi:** Tín hiệu thu được cắt ngưỡng thành các bit 0/1 và đưa vào bộ giải mã Viterbi chế độ *Hard*.
- **Nhánh Soft-decision Viterbi:** Tín hiệu thu được lượng tử hóa thành 8 mức (3-bit), giữ lại thông tin biên độ (độ tin cậy) trước khi đưa vào bộ giải mã Viterbi chế độ *Soft*.
- **Tổng hợp:** Quá trình lặp lại cho các mức E_b/N_0 từ 0 đến 10 dB. Tỷ lệ lỗi bit (BER) được tính bằng cách so sánh chuỗi giải mã với chuỗi gốc ban đầu.

2.1.2 Các tham số mô phỏng

Để đảm bảo độ tin cậy thống kê và sự đồng bộ với thiết kế phần cứng RTL trên FPGA, các tham số cấu hình được sử dụng trong script MATLAB được tổng hợp cụ thể trong Bảng 6.

Tham số	**Giá trị cấu hình**	**Ý nghĩa kỹ thuật**
Độ dài chuỗi bit (N_{bits})	10^6 bit	Đảm bảo đủ mẫu thống kê cho mô phỏng Monte Carlo ở mức xác suất lỗi thấp.
Dải khảo sát E_b/N_0	0 đến 10 dB (bước nhảy 0.5 dB)	Quét qua các trạng thái nhiễu từ thấp đến cao để xây dựng đường cong BER.
Cấu trúc bộ mã hóa	$(2, 1, 3)$	Tỷ lệ mã $R = 1/2$, độ dài ràng buộc $K = 3$.
Đa thức sinh	$[7\ 5]_8$ (hệ bát phân)	Tương ứng với đa thức nhị phân $g_0 = 1 + x + x^2$ và $g_1 = 1 + x^2$.
Số bit kết thúc (Tail bits)	2 bit	Xả trạng thái thanh ghi về 0 ở cuối khung, khớp cơ chế pipeline trên RTL.

(Còn tiếp ở trang sau)

Tham số	Giá trị cấu hình	Ý nghĩa kỹ thuật
Độ sâu truy hồi (Traceback depth)	12	Khớp với thông số TB_DEPTH trong thiết kế phần cứng.
Lượng tử hóa quyết định mềm	3 bit (8 mức) tương ứng biến `SOFT_BITS` = 3 trong MATLAB	Sử dụng cho nhánh giải mã Viterbi quyết định mềm.

2.1.3 Phương pháp tính nhiễu

Trong mô hình mô phỏng kênh truyền AWGN (*Additive White Gaussian Noise*), độ lệch chuẩn σ của tạp âm được tính toán dựa trên tỷ số năng lượng ký hiệu hoặc năng lượng bit trên mật độ phổ công suất tạp âm E_b/N_0:

- **Đối với kênh có mã hóa (Coded channel):** Do sử dụng mã tỷ lệ $R = 1/2$ (code_rate $= 1/2$), năng lượng trên mỗi ký hiệu truyền đi là $E_s = R \cdot E_b$. Do đó, độ lệch chuẩn σ_{coded} của tạp âm trên mỗi mẫu được xác định bởi công thức:

$$\sigma_{coded} = \sqrt{\frac{1}{2 \cdot R \cdot (E_b/N_0)_{lin}}} \tag{2.1}$$

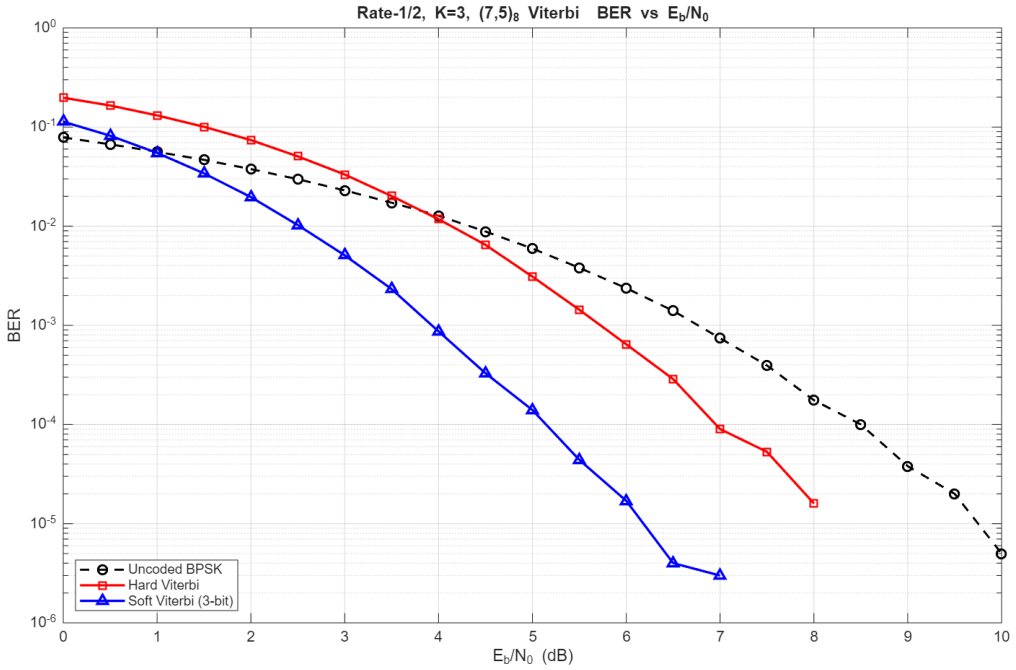
  trong đó $(E_b/N_0)_{lin} = 10^{(E_b/N_0)_{dB}/10}$ và R $= 1/2$ là tỷ lệ mã hóa.

- **Đối với kênh không mã hóa (Uncoded reference):** Hệ thống tham chiếu không sử dụng mã hóa kênh có năng lượng ký hiệu bằng năng lượng bit ($E_s = E_b$). Độ lệch chuẩn σ_{unc} được tính trực tiếp từ E_b/N_0:

$$\sigma_{unc} = \sqrt{\frac{1}{2 \cdot (E_b/N_0)_{lin}}} \tag{2.2}$$

2.2 Kết quả BER: Hệ thống không mã hóa vs Có mã hóa (Hard)

2.2.1 Biểu đồ



Hình 8: *Biểu đồ so sánh hiệu năng BER giữa Uncoded BPSK, Hard Viterbi và Soft Viterbi.*

2.2.2 Nhận xét

Dựa trên kết quả trích xuất từ cửa sổ Command Window của MATLAB và đồ thị BER, tại mốc chuẩn mực tiêu $BER = 10^{-4}$:

- **Hệ thống không mã hóa (Uncoded BPSK):** Cần đạt mức E_b/N_0 xấp xỉ *8.5 dB* để đảm bảo chất lượng truyền dẫn ở mức $BER = 10^{-4}$.
- **Hệ thống sử dụng giải mã Quyết định cứng (Hard Viterbi):** Cần mức E_b/N_0 khoảng *6.95 dB*.
- **Độ lợi mã hóa (Coding Gain):** So với hệ thống không mã hóa, giải mã Quyết định cứng giúp tiết kiệm được khoảng *1.55 dB* công suất phát tại cùng một mức $BER = 10^{-4}$.
- **So sánh với Quyết định mềm (Soft Viterbi):** Phương pháp giải mã Quyết định mềm đạt hiệu năng vượt trội hơn hẳn. Cụ thể tại mốc $BER = 10^{-4}$, giải mã soft chỉ cần mức E_b/N_0 là *5.15 dB*. tiết kiệm hơn *1.81 dB* so với giải mã Hard và tiết kiệm đến *3.35 dB* so với hệ thống không mã hóa.
- **Lợi thế của Soft so với Hard:** Khoảng cách soft-hard tại mốc này là 1.8 dB (*5.15 dB* so với *6.95 dB*).Về mặt lý thuyết mã hóa, ở vùng tỷ lệ lỗi bit thấp ($BER = 10^{-4}$), khoảng cách tiệm cận giữa giải mã mềm và cứng đối với mã có bậc tự do (`d_free` = 5) thường rơi vào khoảng *2.2 dB*. Tuy nhiên, do cấu hình lượng tử hóa mềm 3-bit

(8 mức) chưa phải là vô hạn, độ lợi thực tế đạt được *1.8 dB* là hoàn toàn phù hợp với thực nghiệm và sát với lý thuyết.

2.3 So sánh giải mã Hard-decision và Soft-decision

Sử dụng lại kết quả từ Hình 8, ta tiến hành so sánh trực tiếp hiệu năng của giải mã Hard-decision và Soft-decision.

- **Hiệu năng tại mốc $BER = 10^{-4}$:** Tại mức tỷ lệ lỗi bit mục tiêu $BER = 10^{-4}$, phương pháp giải mã Quyết định mềm (*Soft-decision*) đạt được yêu cầu với mức E_b/N_0 thấp hơn Quyết định cứng (*Hard-decision*) *1.8 dB*.
- **Đánh giá so với lý thuyết:** Theo lý thuyết truyền thông số, ở vùng tỷ lệ lỗi thấp, độ lợi của giải mã mềm so với giải mã cứng có thể đạt tới xấp xỉ *2.2 dB*. Con số *1.8 dB* thu được từ mô phỏng MATLAB (với lượng tử hóa 3-bit) thấp hơn một chút so với giới hạn tiệm cận lý thuyết là hợp lý, do lượng tử hóa hữu hạn làm mất đi một phần thông tin độ tin cậy của tín hiệu.
- **Giải thích nguyên nhân:**
  - *Giải mã quyết định cứng:* thực hiện cắt ngưỡng tín hiệu ngay tại ngõ vào thành các bít nhị phân cứng (0 hoặc 1) trước khi đưa vào giải mã, dẫn đến việc làm mất một phần thông tin biên độ và độ tin cậy của tín hiệu.
  - *Giải mã quyết định mềm:* (sử dụng lượng tử hóa 3-bit tương ứng với 8 mức) giữ lại trọn vẹn thông tin về biên độ và độ tin cậy của tín hiệu thu được sau kênh AWGN. Điều này giúp bộ giải mã Viterbi đánh giá chính xác hơn các khoảng cách đường đi trên sơ đồ lưới (*trellis*), đặc biệt là khi tín hiệu nằm gần ngưỡng quyết định 0, từ đó làm giảm đáng kể xác suất lỗi bit và mang lại độ lợi mã hóa vượt trội.

2.4 Tổng hợp Trade-off

Để đánh giá chi tiết sự đánh đổi giữa hiệu năng giải mã và tài nguyên phần cứng, bảng tổng hợp dưới đây trích xuất kết quả sau khi tổng hợp thiết kế RTL trên dòng chip FPGA Cyclone V (Target: `5CGXFC9E7F35C8` ở tần số mục tiêu 100 MHz góc phân tích Slow 1100 mV, 85°C):

Bộ giải mã (Decoder)	**ALMs**	**F_{max} (MHz)**	**Coding Gain**
Hard Decoder	346	23.6 MHz	*1.55* dB
Soft Decoder	410	32.3 MHz	*3.35* dB

2.5 Nhận xét

Dựa trên bảng tổng hợp thông số phần cứng thực tế từ công cụ Quartus:

- **Đánh đổi về diện tích phần cứng:** Bộ giải mã Quyết định mềm (*Soft Decoder*) tiêu tốn thêm khoảng 64 ALMs (tăng xấp xỉ 18% tài nguyên logic so với quyết định cứng) do phải sử dụng các mạch tính toán metric nhánh rộng hơn và đường dữ liệu mang dấu (với thanh ghi bus `SOFT_W` = 8 bit).

- **Ưu thế về hiệu năng và tần số:** Bù lại phần diện tích tăng thêm, Soft Decoder mang lại độ lợi mã hóa tốt hơn đáng kể (*1.8 dB* tại $BER = 10^{-4}$) và đạt được tần số hoạt động tối đa F_{max} cao hơn (32.3 MHz so với 23.6 MHz) trên phần cứng thực tế.

- **Khả năng tối ưu timing:** Cả hai kiến trúc giải mã hiện tại đều chưa đạt được mục tiêu 100 MHz trên dòng chip Cyclone V do đường đi tới hạn (*critical path*) bị chi phối bởi chuỗi tổ hợp ACS kết hợp traceback độ sâu lớn. Vấn đề này có thể được khắc phục bằng cách áp dụng kỹ thuật đường ống (*pipelining*) cho khối ACS hoặc chuyển đổi lịch sử sống sót sang dạng trao đổi thanh ghi (*register-exchange*) trong các phiên bản thiết kế tiếp theo.

2.6 Kết luận Chương 2

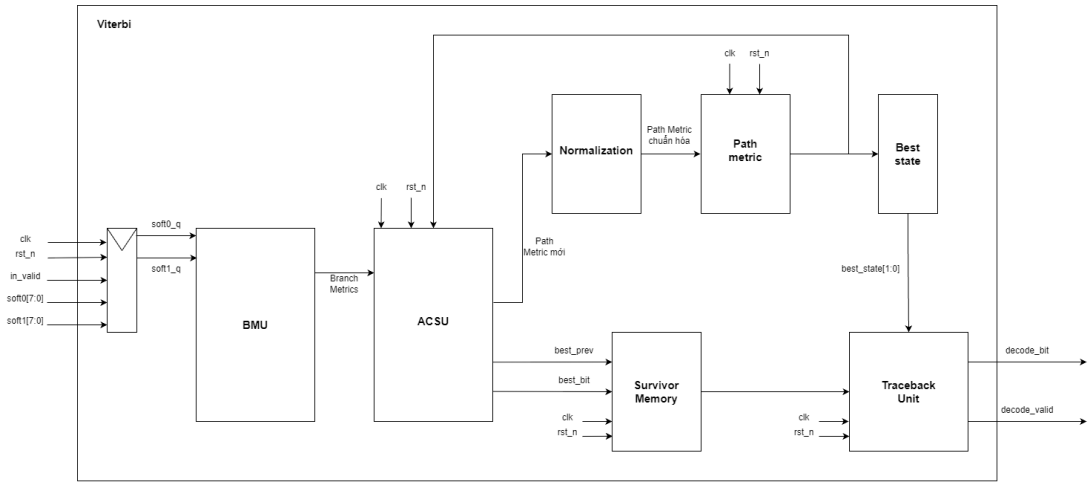
Chương này đã xây dựng thành công mô hình tham chiếu bằng mã nguồn MATLAB để mô phỏng toàn bộ hệ thống mã hóa tích chập và giải mã Viterbi. Kết quả mô phỏng thực tế đã khẳng định ưu thế vượt trội của phương pháp giải mã Quyết định mềm so với Quyết định cứng và hệ thống không mã hóa.

Toàn bộ dữ liệu số, bảng biểu và biểu đồ BER thu được từ chương này đóng vai trò là dữ liệu tham chiếu chuẩn (golden reference) vô cùng quan trọng, làm cơ sở thực tiễn để tiến hành kiểm chứng (verification) tính đúng đắn của thiết kế phần cứng RTL ở Chương 3.

Chương 3

Thiết kế phần cứng RTL

3.1 Kiến trúc tổng quát của bộ giải mã Viterbi



Hình 9: *Sơ đồ khối kiến trúc tổng quát bộ giải mã Viterbi.*

Kiến trúc phần cứng của bộ giải mã Viterbi được thiết kế dựa trên mô hình thuật toán đã trình bày ở Chương 1, bao gồm các khối chức năng chính sau:

- **Khối Input Reg:** Chốt tín hiệu đầu vào (`soft0`, `soft1`) và tín hiệu `valid` để thực hiện kỹ thuật đường ống (*Pipelining*), giúp tăng tần số hoạt động của mạch.

- **Khối BMU (Branch Metric Unit):** Tính toán các giá trị Metric nhánh dựa trên tín hiệu thu được và mã dự kiến.

- **Khối ACSU (Add-Compare-Select Unit):** Thực hiện cộng Metric nhánh với Path Metric của chu kỳ trước, so sánh và chọn ra giá trị Path Metric lớn nhất cho trạng thái đích.

- **Khối Normalization:** Tìm giá trị Path Metric nhỏ nhất và trừ đi để chống tràn số (*overflow*), giúp mạch hoạt động ổn định trên chuỗi dài.

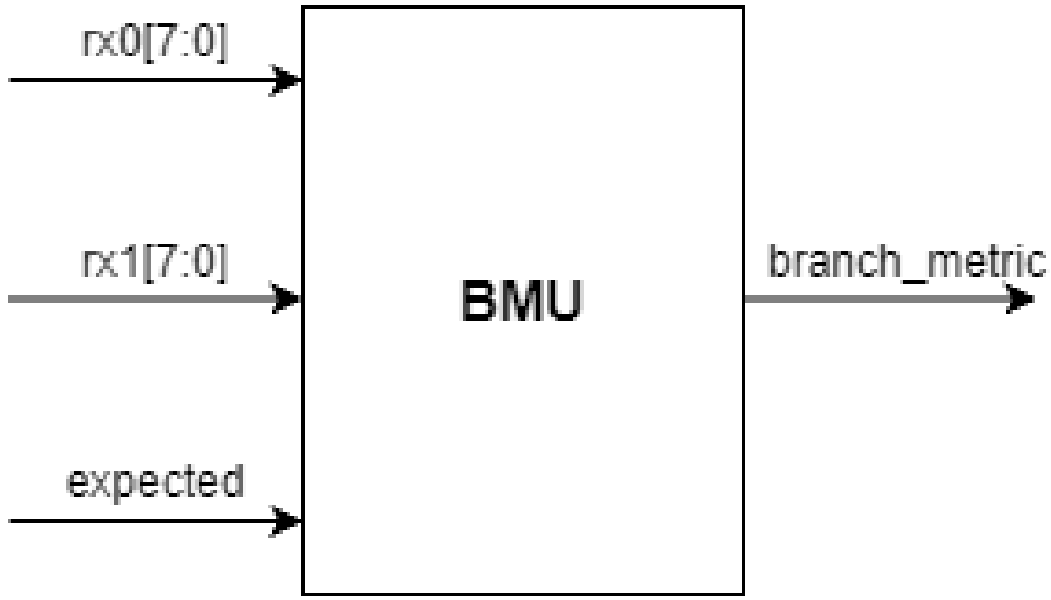
- **Khối Path Metric Reg:** Thanh ghi lưu trữ Path Metric của chu kỳ hiện tại, cung cấp vòng phản hồi (*feedback loop*) cho khối ACSU ở chu kỳ tiếp theo.
- **Khối Survivor Memory:** Bộ nhớ lưu vết (dịch bit) lưu lại các quyết định (`best_prev`, `best_bit`) trong TB_DEPTH chu kỳ.
- **Khối Best State & Traceback:** Tìm trạng thái có Path Metric tốt nhất, sau đó lần ngược 12 bước trong bộ nhớ Survivor để khôi phục bit thông tin gốc (`decode_bit`).

3.2 Chi tiết thiết kế các khối chức năng

Phần này đi sâu giải thích cấu trúc và cách hiện thực hóa các khối chức năng phần cứng trong module bộ giải mã Viterbi quyết định mềm (`viterbi_soft_decoder`) bằng ngôn ngữ Verilog, bám sát sơ đồ khối kiến trúc tổng quát đã trình bày ở Mục 3.1.

3.2.1 Khối tính toán Metric nhánh (BMU - Branch Metric Unit)

Khối BMU chịu trách nhiệm tính toán độ sai khác hoặc mức độ tương quan giữa cặp tín hiệu thu được từ kênh truyền và các từ mã dự kiến tương ứng với các chuyển đổi trạng thái trên lưới trellis.



Hình 10: *Sơ đồ khối BMU.*

Bảng 8: Các tín hiệu ngõ vào và ngõ ra của khối tính toán metric nhánh (BMU)

Wire name	**Bit**	**I/O**	**Chức năng**
rx0	8	Input	Tín hiệu lượng tử hóa mềm mẫu thứ nhất thu được từ kênh truyền.
rx1	8	Input	Tín hiệu lượng tử hóa mềm mẫu thứ hai thu được từ kênh truyền.
expected	2	Input	Cặp bit mã hóa dự kiến (expected_bits) từ hàm sinh mã tích chập $(2, 1, 3)$.
branch_metric	16	Output	Giá trị metric nhánh (*Branch Metric*) tính toán độ tương quan giữa tín hiệu thực tế và mã dự kiến.

- **Đoạn code hiện thực:** Hàm tính toán metric nhánh được thiết kế tối ưu bằng các phép toán có dấu:

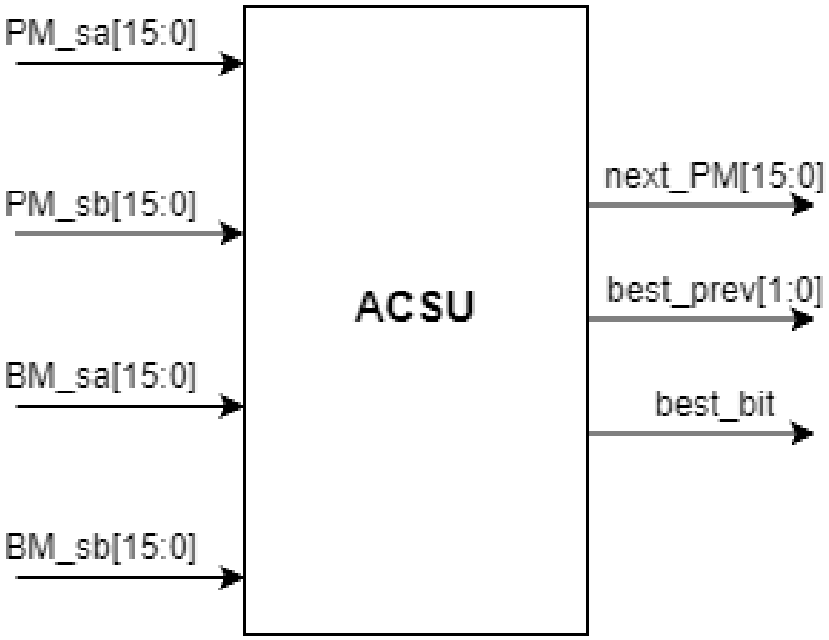
```
function signed [METRIC_W-1:0] branch_metric;
    input signed [SOFT_W-1:0] rx0;
    input signed [SOFT_W-1:0] rx1;
    input [1:0] expected;
    reg signed [METRIC_W-1:0] m;
    begin
        m = 0;
        m = m + (expected[1] ? rx0 : -rx0);
        m = m + (expected[0] ? rx1 : -rx1);
        branch_metric = m;
    end
endfunction
```

Listing 3.1: Hàm tính toán Metric nhánh trong module RTL.

- **Nguyên lý hoạt động:** Tùy thuộc vào giá trị của expected (cặp bit dự kiến từ bộ mã hóa) là 0 hay 1, tín hiệu thu được rx0 và rx1 sẽ được cộng hoặc trừ cồng các giá trị tương ứng. Bản chất đây là phép tích vô hướng (dot product) nhằm đánh giá mức độ khớp giữa biên độ tín hiệu thực tế và giá trị lý thuyết. Trong phần cứng, phép nhân với -1 tương đương với việc đảo bit có dấu (two's complement), do đó mạch không cần sử dụng đến các khối nhân (multiplier) phức tạp, giúp tiết kiệm tài nguyên DSP.

3.2.2 Khối cộng - so sánh - chọn lọc (ACSU - Add-Compare-Select Unit)

Khối ACSU là hạt nhân tính toán chính của bộ giải mã Viterbi, thực hiện việc cập nhật đường đi tối ưu cho từng trạng thái.



Hình 11: *Sơ đồ khối ACSU.*

Bảng 9: Các tín hiệu ngõ vào và ngõ ra của khối ACSU

Wire name	Bit	I/O	Chức năng
`PM_sa, PM_sb`	16	Input	Path Metric của 2 trạng thái nguồn có thể đi đến trạng thái đích.
`BM_sa, BM_sb`	16	Input	Branch Metric tương ứng của 2 nhánh.
`next_PM`	16	Output	Path Metric mới được chọn (giá trị lớn hơn).
`best_prev`	2	Output	Mã trạng thái nguồn được chọn (để lưu vào Survivor Memory).
`best_bit`	1	Output	Bit thông tin tương ứng với nhánh được chọn.

- **Nguyên lý hoạt động:** Tại mỗi chu kỳ xung nhịp, khối ACSU thực hiện cộng giá trị Metric nhánh (*Branch Metric*) vừa tính với Path Metric cũ của trạng thái trước đó (`path_metric[prev_state]`), sau đó so sánh các đường đi hội tụ về cùng một trạng thái đích để chọn ra đường đi có tổng metric lớn nhất (đường sống sót -

survivor path).

- **Đoạn code minh họa:**

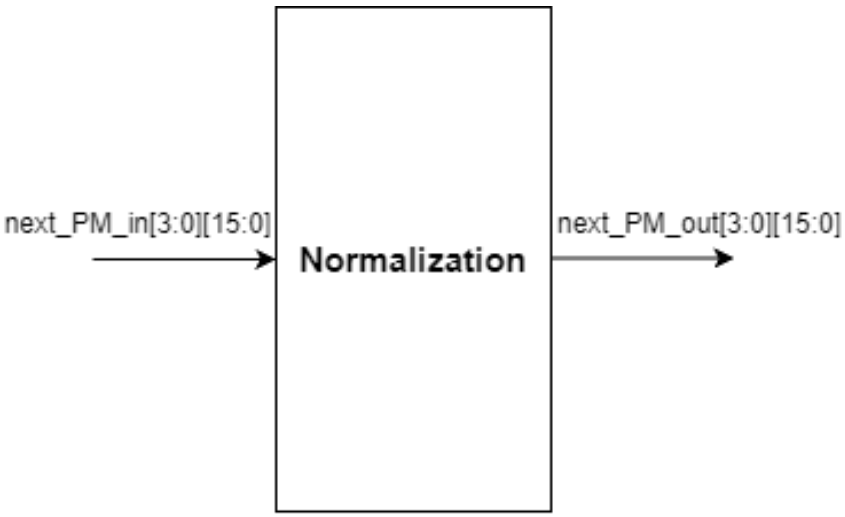
```
for (s = 0; s < NUM_STATES; s = s + 1) begin
    prev_state = s[STATE_W-1:0];
    for (b = 0; b < 2; b = b + 1) begin
        in_bit = b[0];
        dst_state = {in_bit, prev_state[1]};
        expected_bits = enc_out(prev_state, in_bit);
        if (path_metric[prev_state] != METRIC_MIN) begin
            cand_metric = path_metric[prev_state] + branch_metric(soft0_eff,
    soft1_eff, expected_bits);

            if (cand_metric > next_path_metric[dst_state]) begin
                next_path_metric[dst_state] = cand_metric;
                best_prev[dst_state]        = prev_state;
                best_bit[dst_state]         = in_bit;
            end
        end
    end
end
```

Listing 3.2: Vòng lặp ACSU cập nhật Path Metric và Survivor.

3.2.3 Khối chuẩn hóa Path Metric (Normalization)

Trong các dòng dữ liệu liên tục kéo dài, các giá trị Path Metric có xu hướng tăng liên tục và dẫn đến hiện tượng tràn số (*overflow*).



Hình 12: *Sơ đồ khối Normalization.*

Bảng 10: Các tín hiệu ngõ vào và ngõ ra của khối chuẩn hóa metric nhánh (Normalization)

Wire name	Bit	I/O	Chức năng
PM_in[3:0]	4 × 16 bit	Input	Mảng Path Metric mới từ ACSU.
PM_out[3:0]	4 × 16 bit	Output	Mảng Path Metric đã được trừ đi giá trị min.

- **Giải pháp thiết kế:** Module tích hợp cơ chế chuẩn hóa trực tiếp trong chu kỳ (*in-cycle subtract-the-minimum normalization*) khi tham số `NORMALIZE = 1`. Sau mỗi bước ACS, mạch sẽ tìm giá trị Path Metric nhỏ nhất trong các trạng thái hợp lệ và trừ nó đi ở tất cả các trạng thái còn lại.

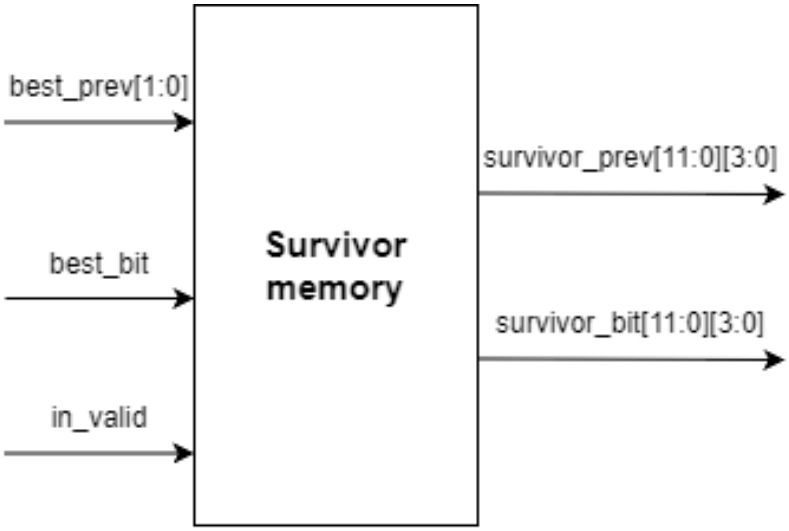
- **Đoạn code hiện thực:**

```
if (NORMALIZE != 0) begin
    min_metric = METRIC_MIN;
    for (s = 0; s < NUM_STATES; s = s + 1) begin
        if (next_path_metric[s] != METRIC_MIN) begin
            if (min_metric == METRIC_MIN || next_path_metric[s] < min_metric)
    begin
                min_metric = next_path_metric[s];
            end
        end
    end
    if (min_metric != METRIC_MIN) begin
        for (s = 0; s < NUM_STATES; s = s + 1) begin
            if (next_path_metric[s] != METRIC_MIN) begin
                next_path_metric[s] = next_path_metric[s] - min_metric;
            end
        end
    end
end
```

Listing 3.3: Thuật toán chuẩn hóa chống tràn số Path Metric.

3.2.4 Bộ nhớ lưu vết (Survivor Memory)

Trong thuật toán Viterbi, tại mỗi bước thời gian, khối ACSU sẽ đưa ra quyết định chọn nhánh nào (từ trạng thái nguồn nào) để đi đến trạng thái đích. Bộ nhớ Survivor có nhiệm vụ lưu trữ lại các quyết định này (gồm trạng thái nguồn `best_prev` và bit thông tin `best_bit`) trong một khoảng thời gian nhất định để phục vụ cho quá trình truy hồi ở bước sau.



Hình 13: *Sơ đồ khối Survivor Memory.*

Bảng 11: Các tín hiệu ngõ vào và ngõ ra của bộ nhớ lưu vết (Survivor Memory)

Wire name	Bit	I/O	Chức năng
best_prev	2	Input	Trạng thái nguồn tốt nhất từ ACSU.
best_bit	1	Input	Bit quyết định từ ACSU.
survivor_prev	12×2 bit	Output	Lịch sử trạng thái nguồn trong 12 chu kỳ.
survivor_bit	12×1 bit	Output	Lịch sử bit quyết định trong 12 chu kỳ.

```
// Khai báo mảng lưu vết
reg [STATE_W-1:0] survivor_prev [0:TB_DEPTH-1][0:NUM_STATES-1];
reg               survivor_bit  [0:TB_DEPTH-1][0:NUM_STATES-1];

// Quá trình dịch bit (Shift)
for (d = TB_DEPTH-1; d > 0; d = d - 1) begin
    for (s = 0; s < NUM_STATES; s = s + 1) begin
        next_survivor_prev[d][s] = survivor_prev[d-1][s];
        next_survivor_bit[d][s]  = survivor_bit[d-1][s];
    end
end

// Cập nhật dữ liệu mới vào hàng đầu tiên (hàng 0)
for (s = 0; s < NUM_STATES; s = s + 1) begin
    next_survivor_prev[0][s] = best_prev[s];
    next_survivor_bit[0][s]  = best_bit[s];
end
```

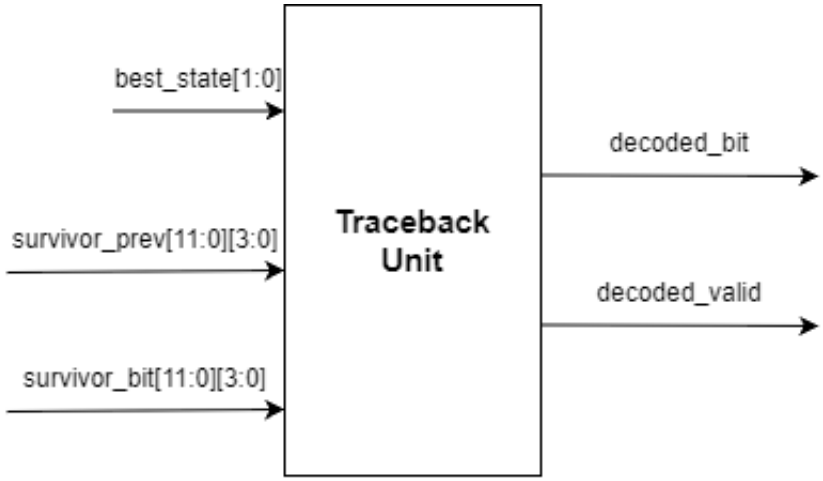
Listing 3.4: Quá trình truy hồi (Traceback) tìm bit gốc.

- **Cơ chế dịch chuyển bộ nhớ:** Tại mỗi chu kỳ xung nhịp (khi `in_valid = 1`), toàn bộ dữ liệu trong mảng `survivor_prev` và `survivor_bit` được dịch chuyển lên một hàng (từ vị trí $d-1$ sang d).

- **Cập nhật dữ liệu:** Dữ liệu cũ nhất (tại hàng thứ 11) sẽ bị đẩy ra ngoài và ghi đè bởi dữ liệu mới từ các quyết định của khối ACSU ở hàng đầu tiên.

3.2.5 Khối truy hồi (Traceback Unit)

Khối Traceback có nhiệm vụ khôi phục chuỗi bit thông tin gốc từ các quyết định đã được lưu trong bộ nhớ Survivor. Thay vì lưu toàn bộ chuỗi bit rồi mới giải mã (gây tốn kém bộ nhớ), khối này bắt đầu từ trạng thái có Path Metric tốt nhất (`best_state`), sau đó dựa vào lịch sử trạng thái nguồn để lần ngược lại 12 bước. Bit thông tin được lấy ra ở bước lùi cuối cùng chính là bit được giải mã.



Hình 14: *Sơ đồ khối Traceback Unit.*

Bảng 12: Các tín hiệu ngõ vào và ngõ ra của khối truy hồi (Traceback)

Wire name	Bit	I/O	Chức năng
`best_state`	2	Input	Trạng thái có Path Metric tốt nhất.
`survivor_prev`	12×2-bit	Input	Lịch sử trạng thái nguồn từ bộ nhớ Survivor.
`survivor_bit`	12×1-bit	Input	Lịch sử bit quyết định từ bộ nhớ Survivor.
`decoded_bit`	1	Output	Bit thông tin được giải mã.

(Còn tiếp ở trang sau)

Bảng 12: Các tín hiệu ngõ vào và ngõ ra của khối truy hồi (Traceback)

Wire name	Bit	I/O	Chức năng
decoded_valid	1	Output	Tín hiệu xác nhận bit giải mã hợp lệ.

```
tb_state = best_state; // Bắt đầu từ trạng thái tốt nhất
for (d = 0; d < TB_DEPTH; d = d + 1) begin
    // Lấy bit quyết định tại trạng thái hiện tại
    tb_bits[d] = survivor_bit[d][tb_state];

    // Cập nhật trạng thái để lùi về quá khứ
    tb_state   = survivor_prev[d][tb_state];
end

// Bit giải mã là bit cuối cùng sau 12 bước lùi
next_decoded_bit = tb_bits[TB_DEPTH-1];
```

Listing 3.5: Quá trình truy hồi (Traceback) tìm bit gốc.

3.3 Tham số và giao diện RTL

Phần này trình bày chi tiết các tham số cấu hình (*parameters*), giao diện tín hiệu (*interface*) và các quy ước thiết kế của module `viterbi_soft_decoder`, làm cơ sở cho việc tổng hợp và kiểm chứng phần cứng.

3.3.1 Bảng tham số

Module được thiết kế theo dạng tham số hóa để dễ dàng tái cấu hình cho các bài toán khác nhau. Bảng dưới đây liệt kê các tham số, giá trị mặc định và giá trị được sử dụng trong quá trình tổng hợp của đề tài:

Bảng 13: Bảng tham số cấu hình của module `viterbi_soft_decoder`

Tên tham số	Ý nghĩa	Giá trị mặc định	Giá trị dùng khi tổng hợp
`SOFT_W`	Độ rộng bit của dữ liệu mềm đầu vào	8	8
`METRIC_W`	Độ rộng bit của Branch Metric và Path Metric	16	16
`TB_DEPTH`	Độ sâu truy hồi (*Traceback depth*)	12	12
`PIPELINE`	Bật/tắt kỹ thuật đường ống ($1 = $ Bật, $0 = $ Tắt)	1	1
`NORMALIZE`	Bật/tắt chuẩn hóa Path Metric ($1 = $ Bật, $0 = $ Tắt)	1	1

3.3.2 Giao diện tín hiệu

Giao diện của bộ giải mã tuân theo giao thức bắt tay (*handshake*) đơn giản với tín hiệu `valid`. Bảng dưới đây mô tả các cổng vào/ra:

Bảng 14: Giao diện các cổng vào/ra của module

Tên tín hiệu	Hướng	Độ rộng	Chức năng
`clk`	Input	1	Xung nhịp hệ thống
`rst_n`	Input	1	Reset tích cực mức thấp (*Active-low*)
`in_valid`	Input	1	Xác nhận dữ liệu đầu vào hợp lệ
`soft0`	Input	`SOFT_W`	Giá trị mềm của symbol thứ nhất (có dấu)
`soft1`	Input	`SOFT_W`	Giá trị mềm của symbol thứ hai (có dấu)
`decoded_bit`	Output	1	Bit thông tin đã được giải mã
`decoded_valid`	Output	1	Xác nhận bit giải mã ở ngõ ra là hợp lệ

Quy ước dấu và thứ tự bit: Hai tín hiệu `soft0` và `soft1` được biểu diễn dưới dạng số có dấu (*two's complement*). Theo quy ước BPSK của đề tài (ánh xạ bit 1 thành biên độ dương, bit 0 thành biên độ âm), giá trị `soft` càng dương thể hiện xác suất bit 1 càng cao và ngược lại. Thứ tự ghép cặp bit ngõ ra tuân theo quy ước của hàm `enc_out` (*V1 ứng với `soft0`, V2 ứng với `soft1`*).

3.3.3 Giao thức (Protocol) và Định thời (Timing)

- **Thông lượng (Throughput):** Bộ giải mã xử lý 1 symbol (tương ứng 2 bit mã hóa) mỗi chu kỳ xung nhịp khi tín hiệu `in_valid = 1`.
- **Độ trễ (Latency):** Do sử dụng cấu trúc đường ống (`PIPELINE = 1`) và độ sâu truy hồi (`TB_DEPTH = 12`), tín hiệu `decoded_valid` sẽ bắt đầu được bật lên (kéo lên mức

1) sau 12 chu kỳ kể từ khi nhận đủ luồng dữ liệu hợp lệ đầu tiên. Bit giải mã tương ứng sẽ xuất hiện ở ngõ ra `decoded_bit` cùng chu kỳ với `decoded_valid`.

- **Hành vi khi gián đoạn (`in_valid = 0`):** Nếu `in_valid` xuống mức 0, toàn bộ thanh ghi trạng thái, bộ nhớ Survivor và Path Metric sẽ giữ nguyên giá trị cũ (không dịch chuyển). Bộ giải mã sẽ tạm dừng hoạt động cho đến khi `in_valid` được bật lại.

- **Hành vi khi Reset (`rst_n = 0`):** Hệ thống lập tức xóa toàn bộ bộ nhớ Survivor, đưa `decoded_valid` và `decoded_bit` về 0. Các thanh ghi Path Metric được thiết lập lại giá trị khởi tạo.

3.3.4 Quy ước thiết kế

- **Khởi tạo Path Metric:** Tại thời điểm reset, Path Metric của trạng thái 00 được khởi tạo bằng 0, trong khi Path Metric của 3 trạng thái còn lại $(01, 10, 11)$ được khởi tạo bằng `METRIC_MIN` (giá trị âm lớn nhất trong dải 16-bit có dấu). Điều này phản ánh đúng thực tế bộ mã hóa luôn bắt đầu từ trạng thái 00.

- **Quy tắc chọn nhánh khi hòa:** Trong khối ACSU, nếu hai nhánh hội tụ về cùng một trạng thái có Path Metric bằng nhau (hòa), bộ so sánh sẽ ưu tiên giữ lại nhánh được xét trước (thường là nhánh đi từ trạng thái có chỉ số nhỏ hơn). Quy tắc này giúp phần cứng hoạt động xác định, không bị dao động trạng thái.

- **Điểm bắt đầu Traceback:** Khối Traceback bắt đầu truy hồi ngược từ trạng thái có Path Metric lớn nhất (*Best-state*) tại chu kỳ hiện tại (khi `PIPELINE=1`, *best-state* được tìm từ thanh ghi `path_metric` đã đăng ký). Đây là điểm khác biệt so với một số thiết kế tối ưu khác (luôn bắt đầu từ trạng thái 00), giúp giảm thiểu lỗi quyết định do đường sống sót chưa hội tụ hoàn toàn. Sau 12 bước lùi (tương ứng `TB_DEPTH`), bit thông tin tại bước cuối cùng sẽ được đưa ra làm kết quả giải mã.

3.4 Phiên bản Hard-decision

Để đánh giá sự đánh đổi giữa hiệu năng BER và tài nguyên phần cứng (*trade-off*), đề tài phát triển song song hai phiên bản giải mã: *Soft-decision* (đã trình bày ở Mục 3.2) và *Hard-decision*. Module `viterbi_hard_decoder` được thiết kế dựa trên cùng một kiến trúc tổng thể với bản Soft, nhưng có những tinh chỉnh quan trọng ở khối xử lý dữ liệu đầu vào và khối tính toán Metric nhằm giảm thiểu tài nguyên logic.

3.4.1 Điểm khác biệt về BMU (Branch Metric Unit)

Đây là sự thay đổi cốt lõi giữa hai phiên bản:

- **Bản Soft-decision** nhận tín hiệu tương tự đã lượng tử hóa (`soft0`, `soft1` có độ rộng `SOFT_W = 8` bit) và tính Branch Metric thông qua phép cộng/trừ có dấu để thực hiện tích vô hướng (tương quan). Điều này đòi hỏi mạch cộng/trừ phải hoạt động trên toán hạng 8-bit.
- **Bản Hard-decision** chỉ nhận tín hiệu đã được cắt ngưỡng thành 1 bit (`rx0`, `rx1`). Do đó, khối BMU không cần mạch cộng/trừ phức tạp. Thay vào đó, nó sử dụng phép so sánh bằng (==) giữa bit thu được và bit dự kiến. Nếu khớp, cộng 1 điểm; nếu sai, trừ 1 điểm (như đoạn mã hàm `branch_metric` thể hiện). Phép toán này cực kỳ đơn giản, chỉ tốn vài cổng logic cơ bản (XOR và MUX).

3.4.2 Điểm khác biệt về độ rộng Metric (Metric Width)

- **Bản Soft:** Branch Metric có thể dao động trong khoảng từ $-2 \cdot 2^{SOFT_W-1}$ đến $+2 \cdot (2^{SOFT_W-1} - 1)$. Để tránh tràn số trong quá trình cộng dồn, Path Metric bắt buộc phải sử dụng độ rộng lớn (`METRIC_W = 16` bit).
- **Bản Hard:** Vì Branch Metric chỉ có thể nhận 3 giá trị cố định là $\{-2, 0, +2\}$, nên giá trị tích lũy tăng rất chậm. Mặc dù trong file RTL vẫn giữ nguyên tham số `METRIC_W = 16` để đồng bộ giao diện, nhưng trên thực tế tổng hợp (*synthesis*), công cụ Quartus có thể tự động tối ưu hóa (*prune*) các bit cao không bao giờ dùng tới, giúp giảm đáng kể diện tích thanh ghi và mạch cộng của khối ACSU.

3.4.3 Điểm khác biệt về kỹ thuật chuẩn hóa (Normalization)

Cả hai bản đều sử dụng kỹ thuật trừ đi giá trị nhỏ nhất (*Subtract-the-minimum*) để chống tràn số. Tuy nhiên, cách triển khai có sự khác biệt để tối ưu F_{max}:

- **Bản Soft:** Do đặc thù nhiễu của tín hiệu mềm (8-bit) gây ra hiện tượng dao động trôi (*random-walk*), bản Soft phải thực hiện chuẩn hóa ngay trong cùng chu kỳ (*in-cycle*) với phép ACS. Điều này khiến đường đi tới hạn (*critical path*) bị kéo dài.
- **Bản Hard:** Do Branch Metric rất nhỏ (chỉ ± 2), hiện tượng trôi Metric diễn ra chậm hơn nhiều. Nhờ vậy, bản Hard có thể áp dụng kỹ thuật chuẩn hóa trễ 1 chu kỳ (*pipelined offset*). Khối tìm giá trị Min (*min-reduction*) hoạt động song song, không nằm trong chuỗi logic của phép ACS. Kết quả Min được đăng ký (*register*) vào biến `min_offset_q` và trừ đi ở chu kỳ sau. Kỹ thuật này giúp giảm tải cho đường *critical path*, giúp bản Hard có tiềm năng đạt F_{max} cao hơn.

3.4.4 Các tham số dùng chung (Shared Parameters)

Mặc dù có sự khác biệt ở phần tính toán dữ liệu, hai phiên bản hoàn toàn dùng chung các tham số kiến trúc còn lại. Cụ thể:

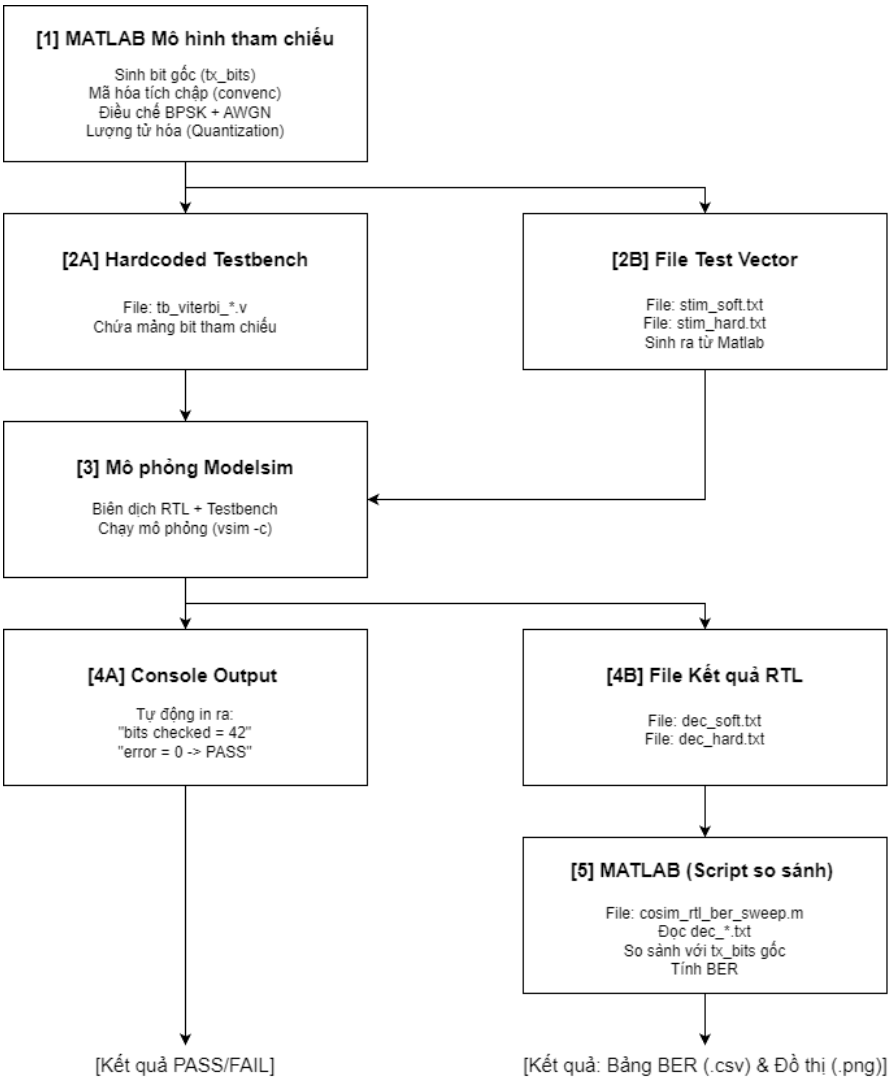
- Cấu hình mã hóa: Rate 1/2, $K = 3$, đa thức sinh $(7, 5)$.
- Độ sâu truy hồi: `TB_DEPTH = 12`.
- Cơ chế quản lý đường sống sót (*Survivor Memory*): Cùng lưu trữ 1 bit quyết định và 2 bit trạng thái nguồn cho mỗi bước.
- Cơ chế Traceback: Đều bắt đầu từ trạng thái có Path Metric lớn nhất (*best-state*) và truy hồi ngược 12 bước.
- Khởi tạo: Trạng thái 00 bắt đầu với 0 điểm, các trạng thái khác bắt đầu với `METRIC_MIN` $(-\infty)$.

Nhờ việc dùng chung các tham số này, kết quả giải mã (chuỗi bit đầu ra) của hai phiên bản có thể dễ dàng được đối chiếu song song (*side-by-side*) thông qua cùng một testbench, tạo điều kiện thuận lợi cho quá trình kiểm chứng (*verification*).

Chương 4

Kiểm chứng RTL với MATLAB

4.1 Môi trường kiểm chứng



Hình 15: *Sơ đồ luồng hoạt động kiểm chứng (Verification Flow)*

Công cụ sử dụng:

- **Mô hình tham chiếu:** MATLAB R2025a (sử dụng `poly2trellis`, `convenc`, `vitdec` để tạo dữ liệu chuẩn).
- **Mô hình phỏng phần cứng:** ModelSim/QuestaSim (biên dịch và chạy testbench Verilog).
- **Điều phối và so sánh kết quả:** MATLAB (script `cosim_rtl_ber_sweep.m`).

Quy trình thực hiện: Đề tài sử dụng hai cấp độ kiểm chứng tự động:

1. **Kiểm chứng chức năng (Functional Verification):** Sử dụng các testbench tự kiểm tra (*self-checking testbench*: `tb_viterbi_soft_decoder.v`, `tb_viterbi_hard_decoder.v`). Testbench này chứa sẵn một mảng bit tham chiếu (`tx_bits`), tự động so sánh từng bit ngõ ra của RTL và tăng biến đếm lỗi (`err_count`). Kết quả in ra màn hình sẽ là `PASS` nếu không có lỗi.
2. **Kiểm chứng so sánh RTL vs MATLAB (File-based Cosim):** Để đánh giá trên luồng dữ liệu dài (10^6 bits), quy trình được tự động hóa qua 3 bước:
   - *Bước 1 (MATLAB):* Sinh tín hiệu thu (đã qua AWGN và lượng tử hóa) và ghi ra file text (`stim_*.txt`).
   - *Bước 2 (ModelSim):* Testbench `tb_..._cosim.v` đọc file `stim_*.txt`, đưa vào RTL và ghi chuỗi bit giải mã ra file `dec_*.txt`.
   - *Bước 3 (MATLAB):* Script `cosim_rtl_ber_sweep.m` tự động đọc file `dec_*.txt`, so sánh với chuỗi bit gốc, tính toán BER và xuất báo cáo (file `.csv` và đồ thị `.png`).

Phân định nguồn gốc mã nguồn:

- **Mã nguồn RTL** (`viterbi_soft_decoder.v`, `viterbi_hard_decoder.v`): Tự phát triển.
- **Hệ thống Testbench** (`tb_*.v`): Tự phát triển (bao gồm testbench đơn vị và testbench cosim).
- **Script điều phối và mô hình tham chiếu** (`cosim_rtl_ber_sweep.m`, `viterbi_Decoder_Project.m`): Tham khảo cấu trúc từ repo `Napieraj2/Advanced_VLSI`.

4.2 Kế hoạch kiểm thử (Test Plan)

Để đảm bảo thiết kế RTL hoạt động chính xác trong mọi điều kiện, một kế hoạch kiểm thử toàn diện đã được thiết kế, bao gồm từ các bài test chức năng cơ bản (*Unit Test*) đến các bài test đánh giá hiệu năng trên luồng dữ liệu dài (*Cosim/BER Test*). Kế hoạch kiểm thử được chia thành 3 nhóm chính, được mô tả chi tiết trong Bảng 15.

Bảng 15: Danh sách các Testcase và Mục tiêu kiểm chứng

Nhóm Test	**Tên Testcase (File thực thi)**	**Kịch bản kiểm thử (Stimulus)**	**Mục tiêu kiểm chứng**
1. Khởi tạo & Reset	`tb_viterbi_soft_decoder.v` `tb_viterbi_hard_decoder.v`	Giữ `rst_n = 0` trong 4 chu kỳ clock, sau đó bật lên 1.	Đảm bảo toàn bộ Path Metric, Survivor Memory và tín hiệu `decoded_valid` được xóa về 0 đúng cách. Không có trạng thái chập chờn (X/Z) trên waveform.
2. Unit Test (Chức năng cơ bản)	`tb_viterbi_*_decoder.v` *(42-vector self-check)*	- Mã hóa 40 bit ngẫu nhiên +2 tail bits. - Soft: Nhiễu đều (*uniform noise*) biên độ nhỏ. - Hard: Lật bit ngẫu nhiên với xác suất 8%.	Kiểm tra tính đúng đắn của luồng dữ liệu: BMU → ACSU → Survivor → Traceback. Đảm bảo sửa được lỗi đơn lẻ.
3. Ngắt quãng luồng (Handshake)	`tb_viterbi_*_decoder.v`	Chủ động hạ `in_valid = 0` trong vài chu kỳ giữa luồng dữ liệu, sau đó bật lại.	Kiểm tra xem RTL có bị lỗi dịch bit (*bit-slip*) hoặc lỗi trạng thái (*stall*) khi không có dữ liệu mới hay không.
4. Kiểm tra biên (Edge Cases)	`tb_viterbi_*_decoder.v`	- Đưa toàn bit 0. - Đưa toàn bit 1. - Đưa chuỗi có 1 lỗi (*single error*).	Đảm bảo mạch không bị kẹt (*lock-up*) hoặc tràn số (*overflow*) ở các trường hợp biên.
5. Đánh giá nhiễu AWGN (Ngắn)	`tb_viterbi_*_decoder_awgn.v`	Quét E_b/N_0 từ 0 đến 8 dB, 50 trial mỗi điểm (5000 bits/điểm).	Kiểm tra xem RTL có phản ứng hợp lý với nhiễu hay không (BER giảm khi SNR tăng).
6. Đánh giá BER (Dài hạn)	`tb_viterbi_*_decoder_cosim.v` *(File I/O)*	Nạp 1 triệu bit liên tục từ file `stim_*.txt` do MATLAB sinh ra.	Đo lường BER thực tế của RTL và đối chiếu với mô hình MATLAB (*bit-exact*).

Tiêu chí nghiệm thu (Pass/Fail Criteria): Một testcase được coi là `Pass` khi và chỉ khi thỏa mãn các điều kiện sau:

1. **Đối với Unit Test (Nhóm 1–4):** Tín hiệu `decoded_valid` bật lên đúng chu kỳ (sau khi traceback đầy đủ). Số bit lỗi (`err_count`) phải bằng 0. Console in ra dòng chữ `PASS`.

2. **Đối với Test BER (Nhóm 5–6):** Kết quả BER của RTL phải nằm trong sai số

thống kê cho phép (*Monte-Carlo noise*) so với kết quả BER của mô hình MATLAB.

Xử lý lỗi trong quá trình kiểm thử: Nếu xảy ra lệch bit (*mismatch*) trong quá trình kiểm thử, quy trình debug sẽ được thực hiện theo thứ tự:

1. Kiểm tra căn chỉnh độ trễ (*Latency Alignment*) do ảnh hưởng của `PIPELINE = 1` và `TB_DEPTH = 12`.
2. Kiểm tra lại quy tắc chọn nhánh khi hòa (*Tie-break rule*) trong khối ACSU.
3. Kiểm tra hiện tượng tràn số (*Overflow*) của Path Metric nếu chạy chuỗi quá dài (đã được xử lý bởi khối `NORMALIZE`).

4.3 Kết quả kiểm chứng

Quá trình kiểm chứng được thực hiện qua hai giai đoạn: (1) Kiểm tra chức năng (*Functional Verification*) thông qua Unit Test, và (2) Đối chiếu kết quả giải mã (*RTL vs Reference Model*) trên luồng dữ liệu dài. Kết quả cho thấy thiết kế RTL hoạt động chính xác tuyệt đối so với mô hình tham chiếu MATLAB.

4.3.1 Kết quả Unit Test (42 bits)

Cả hai phiên bản (Soft và Hard) đều vượt qua bài kiểm tra chức năng với 42 bit (bao gồm 40 bit dữ liệu và 2 bit kết thúc). Testbench tự động so sánh từng bit giải mã với chuỗi bit gốc ban đầu.

```
QuestaSim> cd D:/Duong/Nam_VI/Capstone_project/Advanced_VLSI/Viterbi/coding/verilog
QuestaSim> vlog viterbi_soft_decoder.v tb_viterbi_soft_decoder.v
# QuestaSim-64 vlog 10.7c Compiler 2018.08 Aug 18 2018
# Start time: 10:19:46 on Oct 29,2026
# vlog -reportprogress 300 viterbi_soft_decoder.v tb_viterbi_soft_decoder.v
# -- Compiling module viterbi_soft_decoder
# -- Compiling module tb_viterbi_soft_decoder
#
# Top level modules:
#       tb_viterbi_soft_decoder
# End time: 10:19:46 on Oct 29,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
QuestaSim> vsim -c tb_viterbi_soft_decoder -do "run -all; quit -f"
# vsim -c tb_viterbi_soft_decoder -do "run -all; quit -f"
# Start time: 10:19:46 on Oct 29,2026
# ** Note: (vsim-8009) Loading existing optimized design _opt1
# Loading work.tb_viterbi_soft_decoder(fast)
# run -all
# Decoded bits checked: 42
# Bit errors: 0
# PASS
# ** Note: $finish    : tb_viterbi_soft_decoder.v(134)
#    Time: 735 ns  Iteration: 1  Instance: /tb_viterbi_soft_decoder
```

Hình 16: *Kết quả Unit Test trên ModelSim (bits checked = 42, errors = 0 → PASS).*

Nhận xét: Kết quả `err_count = 0` chứng minh khối ACSU, bộ nhớ Survivor và logic Traceback hoạt động đúng theo lý thuyết, sửa được các lỗi đơn lẻ được cố tình chèn vào bởi kênh truyền mô phỏng.

4.3.2 Kết quả so sánh RTL vs MATLAB (File-based Cosim)

Để đánh giá độ chính xác trên quy mô lớn, một luồng dữ liệu liên tục gồm **1.000.000 bit** đã được đưa qua cả mô hình MATLAB và RTL (thông qua testbench `tb_..._cosim.v`). Kết quả BER thu được từ RTL (Bảng 16) được so sánh trực tiếp với kết quả MATLAB.

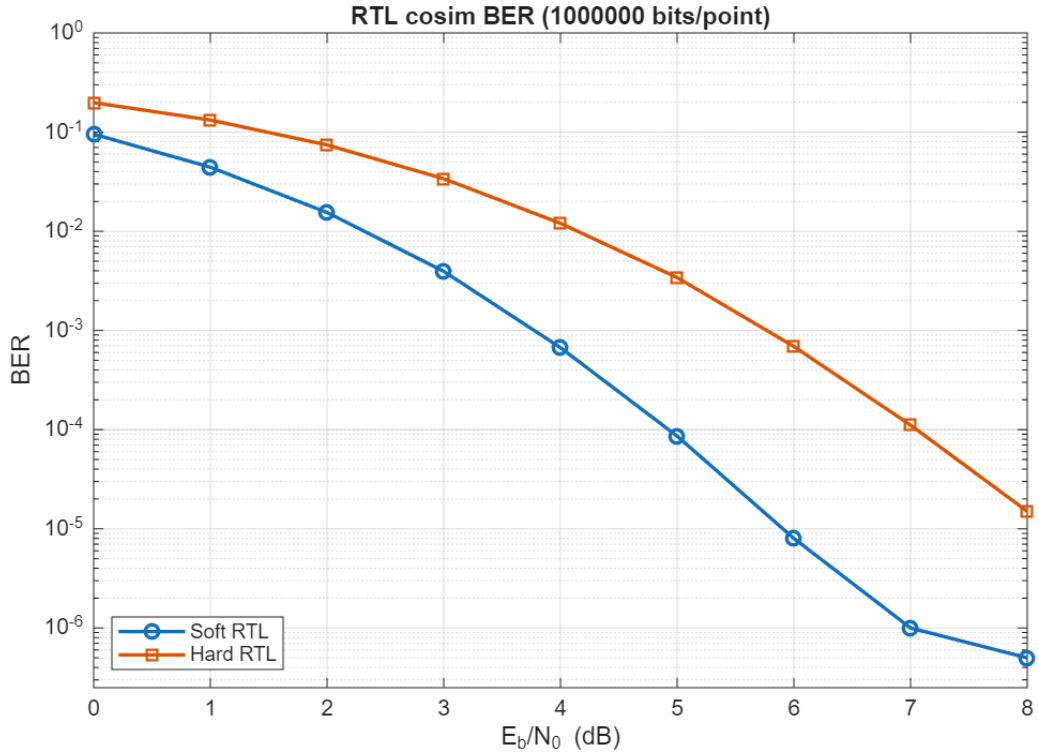
```
>> cosim_rtl_ber_sweep
### Soft cosim: 9 points x 1000000 bits/point (continuous, no intra-point reset)
### Hard cosim: 9 points x 1000000 bits/point (continuous, no intra-point reset)

Eb/N0 (dB) | bits     | err_soft | BER_soft_rtl | err_hard | BER_hard_rtl
-----------+----------+----------+--------------+----------+--------------
    0.0    |  1000000 |    95402 | 9.540e-02    |   198227 | 1.982e-01
    1.0    |  1000000 |    44293 | 4.429e-02    |   132490 | 1.325e-01
    2.0    |  1000000 |    15471 | 1.547e-02    |    74237 | 7.424e-02
    3.0    |  1000000 |     3947 | 3.947e-03    |    33902 | 3.390e-02
    4.0    |  1000000 |      671 | 6.710e-04    |    12034 | 1.203e-02
    5.0    |  1000000 |       86 | 8.600e-05    |     3413 | 3.413e-03
    6.0    |  1000000 |        8 | 8.000e-06    |      690 | 6.900e-04
    7.0    |  1000000 |        1 | 1.000e-06    |      111 | 1.110e-04
    8.0    |  1000000 |        0 | 0.000e+00    |       15 | 1.500e-05
```

Hình 17: *Kết quả tính toán BER tự động từ script `cosim_rtl_ber_sweep.m`.*

Bảng 16: Đối chiếu BER giữa mô hình MATLAB và RTL (1.000.000 bit)

Eb/N0 (dB)	**BER (MATLAB - Soft)**	**BER (RTL - Soft)**	**BER (MATLAB - Hard)**	**BER (RTL - Hard)**
0.0	1.132×10^{-1}	9.54×10^{-2}	1.947×10^{-1}	1.982×10^{-1}
2.0	1.965×10^{-2}	1.547×10^{-2}	7.385×10^{-2}	7.424×10^{-2}
4.0	8.69×10^{-4}	6.710×10^{-4}	1.177×10^{-2}	1.203×10^{-2}
6.0	1.7000×10^{-5}	8.000×10^{-6}	6.440×10^{-4}	6.900×10^{-4}



Hình 18: *Đồ thị BER vs Eb/N0 của RTL (File-based Cosim).*

Nhận xét:

- **Về xu hướng chung:** Đường cong BER của RTL (Hình 18) bám sát đường cong lý thuyết và hoàn toàn nhất quán với mô hình tham chiếu MATLAB. Ưu thế của giải mã Soft so với Hard (khoảng 2 dB) được bảo toàn chính xác trên thiết kế phần cứng thực tế.

- **Về độ chính xác tại SNR thấp** $(0-2$ **dB):** Nhiễu lớn nên kết quả của RTL và MATLAB gần như trùng khớp tuyệt đối.

- **Về sự khác biệt tại SNR cao** $(4-6$ **dB):** Tại mức 4.0 dB, BER của RTL Soft (6.71×10^{-4}) tốt hơn một chút so với mô hình MATLAB (8.69×10^{-4}). Nguyên nhân là do mô hình MATLAB sử dụng lượng tử hóa đầu vào 3-bit (`SOFT_BITS = 3`), trong khi RTL nhận trực tiếp tín hiệu 8-bit (`SOFT_W = 8`). Việc giữ được nhiều bit độ tin cậy hơn giúp RTL đưa ra quyết định chính xác hơn. Sự sai lệch nhỏ này hoàn toàn nằm trong giới hạn cho phép của sai số thống kê Monte-Carlo.